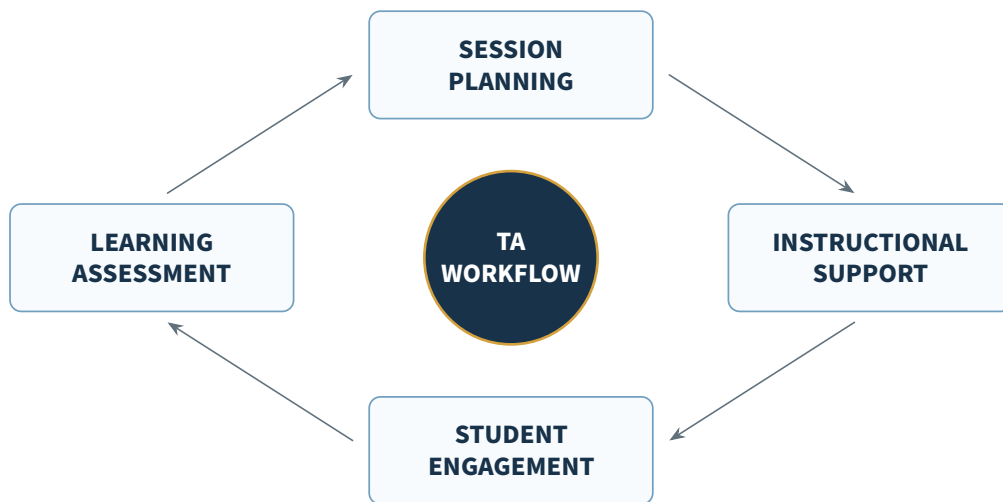




Data Communication & Networking

Teaching Assistant Guide



COURSE OUTLINE

Built from the supplied laboratory materials for Wireshark, eNSP/Huawei VRP, routing, FTP, DHCP, LACP, VLANs, and WAN configuration.

PREPARED BY

Reeyan Afzal Khan

Graduate Research Assistant

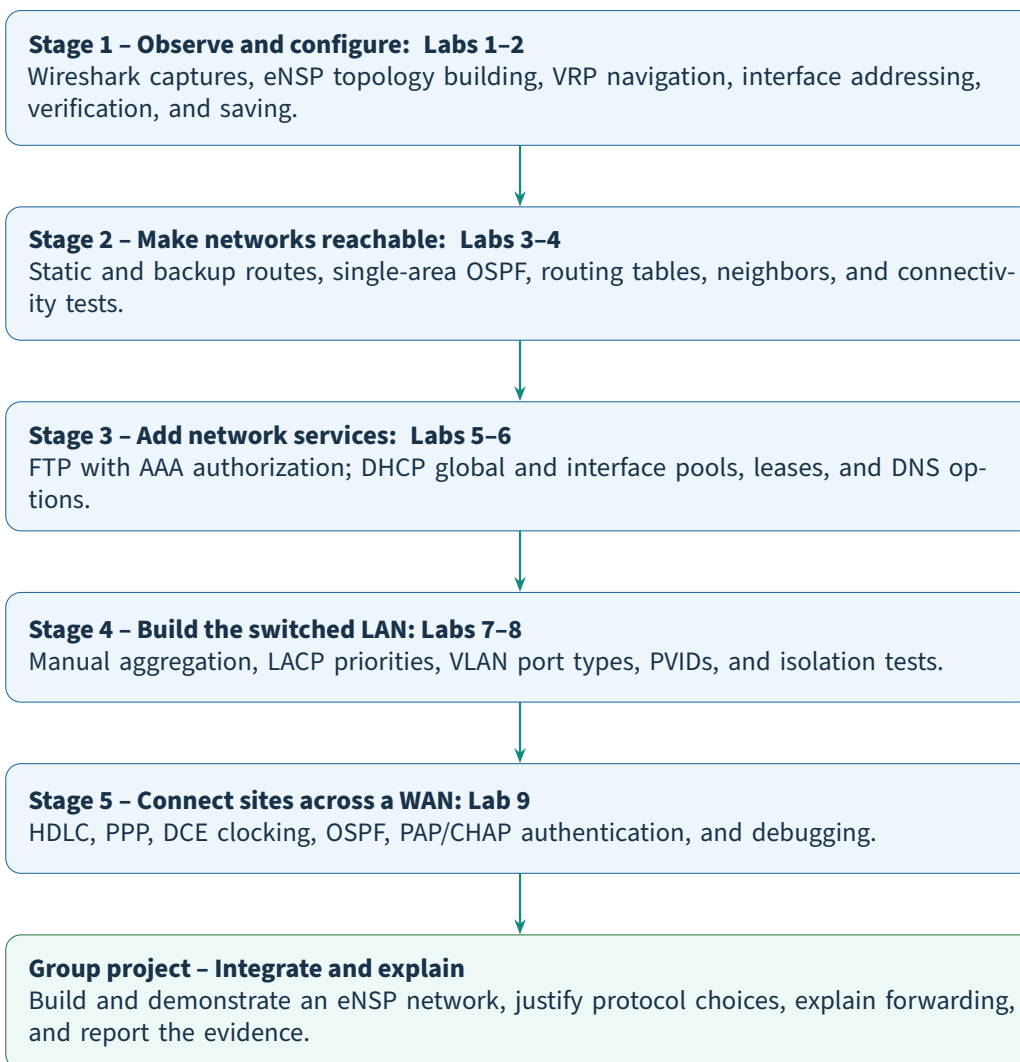


Learning Path

Course Map: From Packets to WANs	ii
Lab 1 Packet Sniffing and Wireshark	1
Lab 1 of 9 — Capture, filter, and read a live conversation	5
Lab 2 eNSP and Huawei VRP Fundamentals	8
Lab 2 of 9 — Build a topology and survive the VRP command line	11
Lab 3 Static Routing	14
Lab 3 of 9 — Static routing and the backup route	18
Lab 4 Dynamic Routing with OSPF	21
Lab 4 of 9 — OSPF: let the network work it out	25
Lab 5 FTP Service and File Transfer	28
Lab 5 of 9 — FTP: a service on top of a working network	32
Lab 6 DHCP Address Allocation	35
Lab 6 of 9 — DHCP: addresses without a spreadsheet	40
Lab 7 Ethernet Link Aggregation and LACP	43
Lab 7 of 9 — Link aggregation: several cables, one link	48
Lab 8 VLAN Configuration: Access, Trunk, and Hybrid Ports	51
Lab 8 of 9 — VLANs: one switch, several networks	56
Lab 9 WAN Links: HDLC, PPP, PAP, and CHAP	59
Lab 9 of 9 — WAN links: HDLC, PPP, and authentication	64
Group Project: Integrated eNSP Network	67
Command Quick Reference	69

Course Map: From Packets to WANs

The labs progress from packet observation to device configuration, routing, services, Layer-2 switching, and WAN protocols.



How to use these notes

Each chapter presents the source material in four forms:

- **Source visuals** show key topologies, interfaces, and protocols.
- **Command blocks** preserve the Huawei VRP configuration sequence.
- **Verification evidence** identifies the state, capture, or test that proves success.
- **Troubleshooting order** provides a repeatable diagnostic workflow.

Key idea

Use the same loop in every lab: **build** → **configure** → **inspect** → **test** → **explain**. Only the protocol changes.

Packet Sniffing and Wireshark

Lab at a glance

Before you start: A basic understanding of hosts, switches, IP traffic, and why packets move through an interface.

By the end, you can: Define packet sniffing and a network traffic trace, navigate the Wireshark interface, capture packets, and connect a packet-list entry to protocol details and raw bytes.

1.1 Purpose and core terms

Before configuring a network, you need a way to observe its traffic. This lab introduces three linked ideas:

Packet sniffing observing packets as they cross a network.

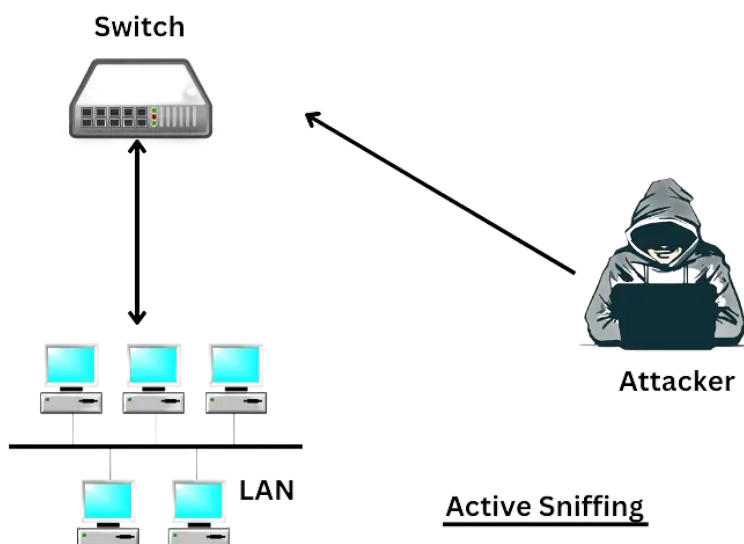
Network traffic trace a time-ordered record of packets seen by an interface.

Wireshark a graphical analyzer for inspecting and troubleshooting packet traffic.

Concept map

Generate traffic → **capture on the correct interface** → **find the relevant packet** → **decode protocol fields** → **relate the packet to the action that produced it.**

Source visual — Packet-sniffing illustration from the supplied slide

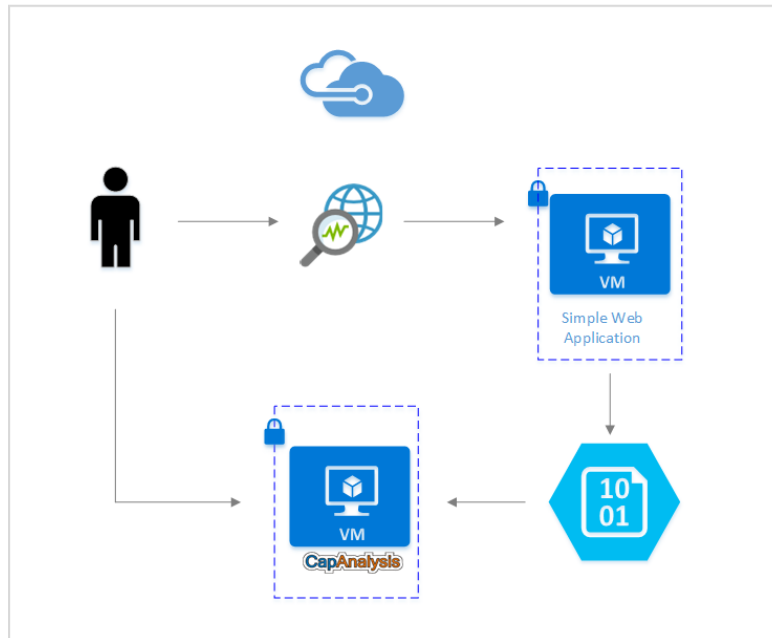


Capture placement matters: the chosen interface must carry the traffic being investigated.

1.2 A traffic trace is evidence, not just a screenshot

A trace records the sequence of events. If a ping, adjacency, or application session fails, it shows what was sent and whether a reply returned.

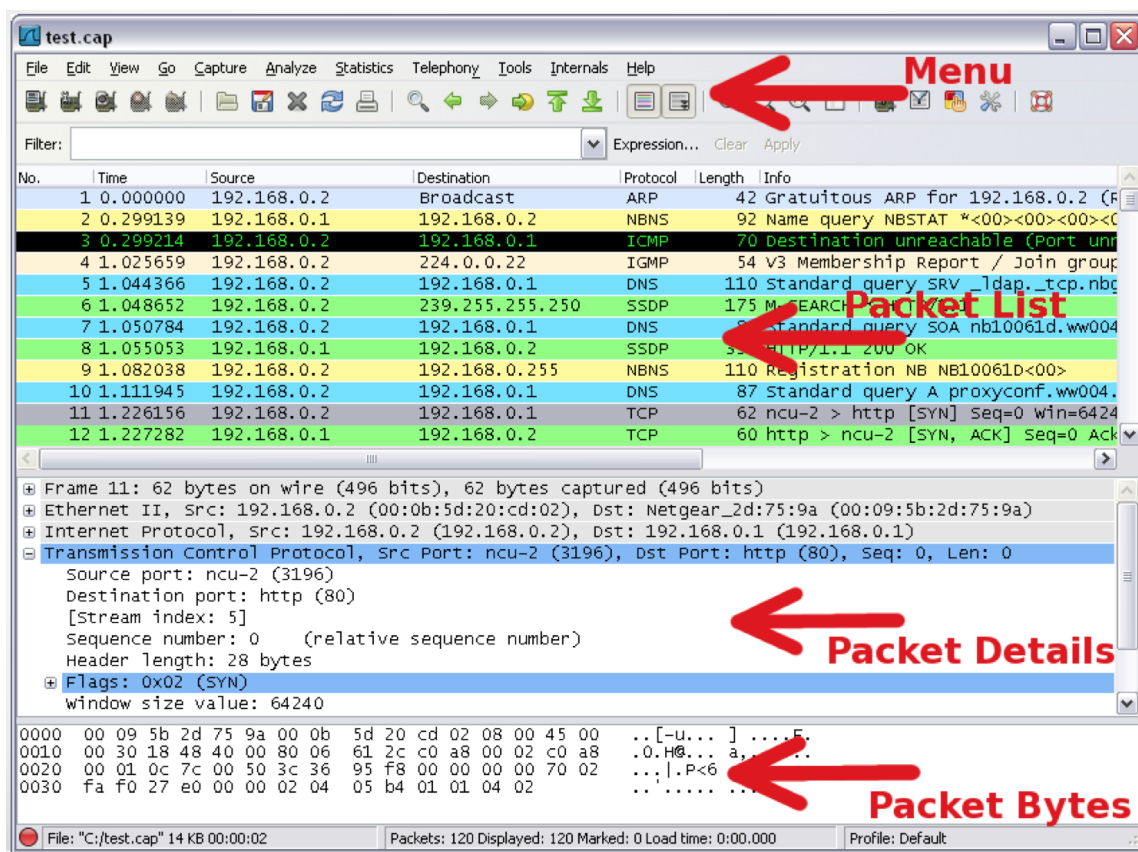
Source visual — Network-traffic trace concept used in the source material



1.3 The Wireshark workspace

Wireshark versions differ visually, but the three-pane workflow is consistent.

Source visual — Wireshark packet list, packet details, and packet bytes panes



Pane

What you use it for

Packet list

One row per frame, with time, endpoints, protocol, length, and summary.

Packet details

Decoded protocol layers, addresses, flags, and fields for the selected frame.

Packet bytes

Raw bytes in hexadecimal and ASCII.

1.4 A disciplined capture workflow

- Step 1:** Identify the traffic you want to observe and the interface that should carry it.
- Step 2:** Start packet capture on that interface.
- Step 3:** Generate a small, known event such as a ping so you have a clear reference point.
- Step 4:** Find the corresponding packet in the packet list using addresses, protocol, and timing.
- Step 5:** Expand the relevant protocol layers in Packet Details.
- Step 6:** Consult Packet Bytes only when byte-level evidence is needed.
- Step 7:** Stop after collecting enough evidence; small traces are easier to analyze.

Worked configuration

If Host A pings Host B, locate the ICMP request and reply. Confirm that the endpoints reverse direction. The trace should prove who sent the request and whether the peer answered.

1.5 What to record in a lab submission

A useful screenshot shows the selected packet, relevant decoded fields, and filter expression. Explain what the evidence proves.

Verification evidence

For this lab, convincing evidence is a capture in which you can point to a known traffic event and identify at least the source, destination, protocol, and the relevant request/reply relationship.

Common mistake

A capture from the wrong interface, or one buried in unrelated traffic, is poor evidence. Generate controlled traffic and filter by protocol or address.

Check your understanding

1. What is the difference between the packet list and packet details panes?
2. Why is a traffic trace easier to interpret when you deliberately generate known traffic?
3. Which pane would you use when you need the exact bytes carried in a frame?
4. If no ping packet appears, which should you question first: the capture location or the packet bytes?

Lab recap

Wireshark organizes a traffic trace into packet summaries, decoded fields, and raw bytes. This evidence supports verification and troubleshooting throughout the course.

PRACTICAL MILESTONE

Lab 1 of 9

Capture, filter, and read a live conversation

Turn a packet capture into evidence

Coverage. Chapter 1.

Goal. Produce a capture that answers a specific question, rather than a screenshot that shows traffic existed.

How to use this lab

Capture on a network you are authorised to observe — your own machine, your own lab segment, or the eNSP simulation. Capturing on a shared campus or corporate network without permission is a disciplinary matter and in many jurisdictions an offence.

Everything below works on loopback or your own interface, so there is no reason to point Wireshark at anyone else's traffic.

Part A — Ask the question first.

The single most common mistake is starting the capture before deciding what it is meant to prove. You then face 40,000 packets and no criterion for which matter.

Write the question down before you press start. Good questions look like:

- Did the DNS lookup for `example.com` succeed, and how long did it take?
- Did the TCP handshake complete, or was it reset?
- Which host sent the first packet of this conversation?

Key idea

A capture filter decides what is *recorded*. A display filter decides what is *shown*. They use different syntaxes, and confusing them is the most common source of an empty result window.

Capture filter (BPF): `tcp port 80`

Display filter (Wireshark): `tcp.port == 80`

Capture broadly and filter afterwards. A capture filter that was too narrow cannot be widened once the traffic has passed.

Part B — A disciplined capture.

Step 1: Choose the interface carrying the traffic. If unsure, watch the sparklines in the Wireshark start screen.

Step 2: Start the capture, then generate the traffic — not the other way round.

Step 3: Perform exactly one transaction. One page load, one ping sequence, one login.

Step 4: Stop the capture immediately. Ten seconds of relevant traffic beats ten minutes of noise.

Step 5: Save the raw `.pcapng` before filtering. That file is your evidence.

Part C — Filters that earn their keep.

Command map

Display filter	Shows
<code>ip.addr == 192.168.1.10</code>	Everything to or from one host
<code>tcp.port == 80 tcp.port == 443</code>	Web traffic in both directions
<code>tcp.flags.syn == 1 && tcp.flags.ack == 0</code>	Connection attempts only
<code>tcp.analysis.retransmission</code>	Packets Wireshark believes were resent
<code>dns</code>	Name resolution
<code>icmp</code>	Ping and unreachable messages
<code>http.request.method == "GET"</code>	Page requests

Common mistake

`ip.addr != 192.168.1.10` does not mean what it appears to. A packet has both a source and a destination address, so the expression is true whenever *either* differs — which is nearly always.

Write `!(ip.addr == 192.168.1.10)` instead. The same trap applies to every address and port field.

Part D — Read one TCP handshake end to end.

Filter to a single conversation (right-click a packet, then *Follow* → *TCP Stream*) and identify the three-way handshake:

No.	Source	Destination	Proto	Info
1	192.168.1.10	93.184.216.34	TCP	49152 > 80 [SYN] Seq=0 Win=64240
2	93.184.216.34	192.168.1.10	TCP	80 > 49152 [SYN, ACK] Seq=0 Ack=1
3	192.168.1.10	93.184.216.34	TCP	49152 > 80 [ACK] Seq=1 Ack=1

Answer from your own capture: which host initiated, what ephemeral port did it choose, and how long elapsed between packets 1 and 3? That last figure is the round-trip time, and it is the number a network engineer actually cares about.

When it does not work

No packets at all. Wrong interface, or a capture filter with a typo. Clear the capture filter and try again.

Traffic appears but not yours. You are probably on a switched network seeing only broadcast and your own frames — which is normal and correct.

HTTPS shows no readable content. That is TLS doing its job. You can still see the handshake, the SNI hostname, and the timing. Analyse those instead.

Try it yourself

Capture a ping to an address that does not exist on your subnet. No reply arrives — so what *does* the capture contain, and what does that tell you about how the host tried to find the destination? Look for ARP.

Verification evidence

- The raw .pcapng is saved and submitted, not only screenshots.
- The question the capture answers is stated before the evidence.
- Display filters are quoted exactly as used.
- Packet numbers are cited when referring to specific packets.
- The capture was taken on a network you are authorised to observe.

Lab deliverable

Submit the .pcapng file, the question it answers, the display filter used, an annotated screenshot of the three-way handshake with packet numbers, and the measured round-trip time. Add one sentence on why capturing broadly and filtering afterwards is the safer order.

eNSP and Huawei VRP Fundamentals

Lab at a glance

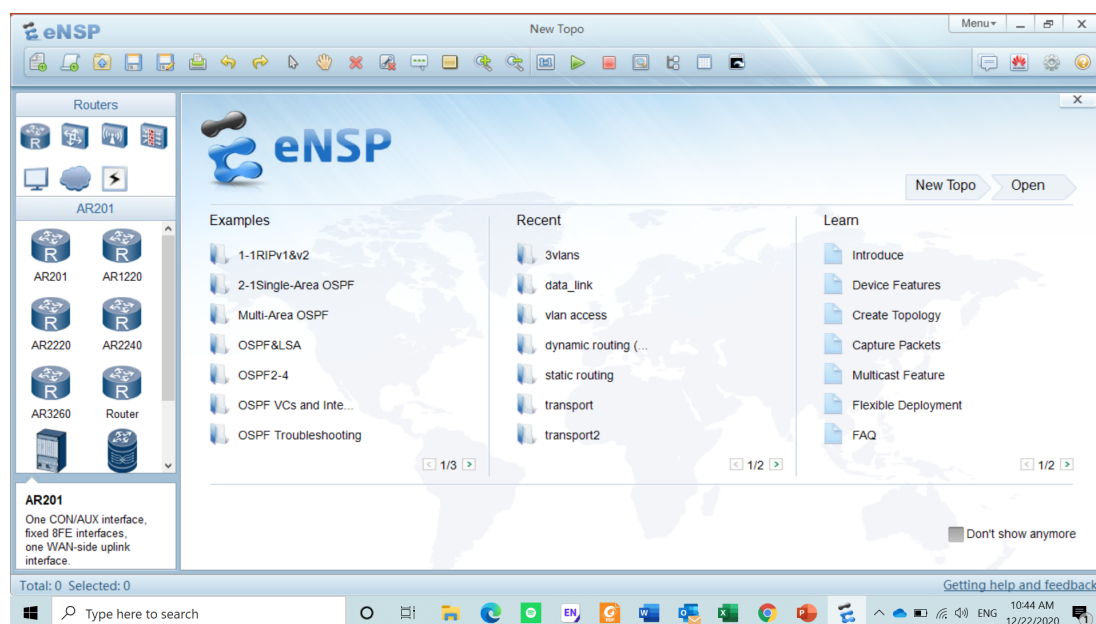
Before you start: Lab 1 packet observation and basic IPv4 addressing concepts.

By the end, you can: Build and start an eNSP topology, capture packets from an eNSP interface, navigate Huawei VRP views, name devices, configure router interfaces, verify state, ping a neighbor, and save a configuration.

2.1 eNSP workflow

eNSP provides a visual environment for building and testing IP networks: place devices, connect interfaces, configure them, start the topology, generate traffic, and inspect the result.

Source visual — eNSP main workspace shown in the supplied Lab 2A material



2.2 The topology-building loop

Step 1: Drag the required hosts, routers, and switches onto the canvas.

Step 2: Connect the specified interfaces with the correct medium.

Step 3: Configure addresses through host settings or device consoles.

Step 4: Start the devices and wait for them to finish booting.

Step 5: Start Wireshark on the interface to be monitored.

Step 6: Generate traffic, such as ping <IP-address>, and inspect it.

Key idea

Use this cycle throughout the course: **build** → **configure** → **inspect** → **test** → **save**.

2.3 Huawei VRP views

VRP commands are organized into hierarchical views; the prompt identifies the active view.

Prompt pattern	Meaning
<Huawei>	User view; commonly used for display, ping, save, and entry into system view.
[Huawei] or [R1]	System view; used for device-wide configuration and entry into protocol/interface views.
[R1-GigabitEthernet0/0/0]	Interface view; commands affect the selected interface.
[R1-ospf-1]	Protocol/process view; commands affect the selected routing process.

2.4 Direct-link practice

R1 and R3 share 10.0.13.0/24, using 10.0.13.1 and 10.0.13.3, respectively.

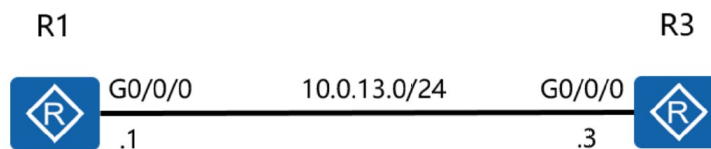
Source visual — Directly connected R1–R3 topology from the supplied Lab 2B slide

Figure 2.1 Lab topology for basic VRP navigation and operation.

2.4.1 Configure R1

```

<Huawei> system-view
[Huawei] sysname R1
[R1] interface GigabitEthernet 0/0/0
[R1-GigabitEthernet0/0/0] ip address 10.0.13.1 24
[R1-GigabitEthernet0/0/0] description This interface connects to R3-G0/0/0

```

2.4.2 Configure R3

```

<Huawei> system-view
[Huawei] sysname R3
[R3] interface GigabitEthernet 0/0/0
[R3-GigabitEthernet0/0/0] ip address 10.0.13.3 255.255.255.0
[R3-GigabitEthernet0/0/0] description This interface connects to R1-G0/0/0

```

The prefix 24 and mask 255 . 255 . 255 . 0 are equivalent.

2.5 Verification commands

Command map

Command	Evidence
<code>display this</code>	Configuration in the current view.
<code>display interface GigabitEthernet 0/0/0</code>	Detailed state, address, description, MTU, and counters.
<code>display ip interface brief</code>	Interface addresses and physical/protocol states.
<code>ping 10.0.13.3</code>	Layer-3 reachability from R1 to R3.
<code>save</code>	Store the current configuration.
<code>display saved-configuration</code>	Stored configuration.

Verification evidence

A healthy direct link has compatible addresses and reports both physical and protocol status as **up**. Verify it with ping, then save the configuration.

2.6 Command abbreviations from the lab

Use abbreviations only after learning the full commands.

Full command	Abbreviation shown
<code>system-view</code>	<code>sys</code>
<code>interface GigabitEthernet 0/0/0</code>	<code>int g 0/0/0</code>
<code>ip address 10.0.13.1 24</code>	<code>ip add 10.0.13.1 24</code>
<code>display</code>	<code>disp</code>
<code>quit</code>	<code>q</code>
<code>save</code>	<code>save</code>

Common mistake

A failed ping is only a symptom. Check interface state first: physical down indicates a link problem; physical up/protocol down points to configuration or peer state. Investigate routing only after the direct link is healthy.

Try it yourself

Build the small switch-and-two-PC topology requested in the Lab 2B material. Assign compatible IPv4 addresses, use `ipconfig` on both PCs, verify connectivity with `ping`, and capture a readable screenshot of the command panel showing the result.

Lab recap

eNSP supplies the virtual network and VRP the CLI. Navigate by the prompt, verify each change, test connectivity, and save.

PRACTICAL MILESTONE

Lab 2 of 9

Build a topology and survive the VRP command line

Two routers, one cable, and a working ping

Coverage. Chapter 2.

Goal. Learn the view hierarchy well enough that every later lab becomes typing rather than guessing.

How to use this lab

This lab has almost no networking content, and it is the most important one in the course. Every configuration you write for the next seven labs assumes you can move between VRP views without thinking about it.

Do it until it is boring. Then do the troubleshooting section, which is where the actual learning is.

Part A — The view hierarchy.

VRP puts you in one of several **views**, and a command only exists in the view it belongs to. “Command not found” almost always means “right command, wrong view”.

```
<Huawei>                                user view      -- look, do not change
<Huawei> system-view
[Huawei]                                system view    -- global configuration
[Huawei] interface GigabitEthernet 0/0/0
[Huawei-GigabitEthernet0/0/0] interface view -- this interface only
[Huawei-GigabitEthernet0/0/0] quit
[Huawei] quit
<Huawei>
```

Key idea

The prompt tells you where you are, and it is the only reliable indicator:

<angle brackets> — user view. Read-only commands: display, ping, save.

[square brackets] — a configuration view. The text after the hyphen names the exact object you are editing.

If a command is rejected, read the prompt before re-reading the command.

Part B — Build the direct link.

Place two routers, connect GE0/0/0 to GE0/0/0, and start both devices. Wait for the interface indicators to turn green before configuring — eNSP will accept configuration on a booting device and then discard it.

```
system-view
sysname R1
interface GigabitEthernet 0/0/0
 ip address 192.168.1.1 24
 undo shutdown
quit
```

Repeat on R2 with 192.168.1.2 24. Then, from R1:

```
<R1> ping 192.168.1.2
```

Common mistake

`ip address 192.168.1.1 24` uses a prefix length. `ip address 192.168.1.1 255.255.255.0` uses a mask. Both are accepted and mean the same thing — but mixing the two styles across a lab report makes it hard to spot a genuine mismatch. Pick one and use it everywhere. This course uses prefix length.

Part C — Verify, never assume.

Configuration entered without error is not configuration that works.

Command map

Command	Answers
<code>display ip interface brief</code>	Which interfaces are up, with what addresses
<code>display current-configuration</code>	What is running right now
<code>display saved-configuration</code>	What survives a reboot
<code>display version</code>	VRP version and uptime
<code>display interface GigabitEthernet 0/0/0</code>	Errors, duplex, and packet counters

The first of these is the one you will use hundreds of times:

```
<R1> display ip interface brief
Interface      IP Address/Mask  Physical  Protocol
GigabitEthernet0/0/0  192.168.1.1/24  up        up
GigabitEthernet0/0/1  unassigned      down      down
```

Key idea

Two status columns, two different failures.

Physical down means the cable, the port, or shutdown. It is a layer-1 problem and no amount of IP configuration will fix it.

Physical up, protocol down means the link is electrically fine but the layer-2 or layer-3 configuration disagrees — a missing address, an encapsulation mismatch, or a failed PPP authentication as in Lab 9.11.

Reading these two columns correctly will diagnose most of the faults you meet this semester.

Part D — Save your work.

```
<R1> save
The current configuration will be written to the device. Continue? [Y/N] y
```

When it does not work

Ping fails but both interfaces are up/up. Check the addresses are in the same subnet.

192.168.1.1/24 and 192.168.2.2/24 are two different networks and will never ping directly.

Device will not start in eNSP. eNSP needs VirtualBox. If routers stay red, VirtualBox is missing, the wrong version, or virtualisation is disabled in BIOS.

Configuration vanished after restart. You did not save. `display saved-configuration` shows what would actually survive.

Command rejected in the right view. Check the abbreviation is unambiguous — `dis` works, `d` does not.

Try it yourself

Run `shutdown` on R1's GE0/0/0, then check `display ip interface brief` on *both* routers. Which columns change on R1, and which on R2? Explain why the far end can tell.

Verification evidence

- `display ip interface brief` shows up/up on both routers.
- A successful ping is captured with its statistics line, not just the prompt.
- The configuration was saved and confirmed with `display saved-configuration`.
- Address style (prefix length or mask) is consistent throughout.

Lab deliverable

Submit the running configuration of both routers, `display ip interface brief` output from each, a successful ping with its statistics, and a short note explaining the difference between the Physical and Protocol columns using your own `shutdown` experiment as evidence.

Static Routing

Lab at a glance

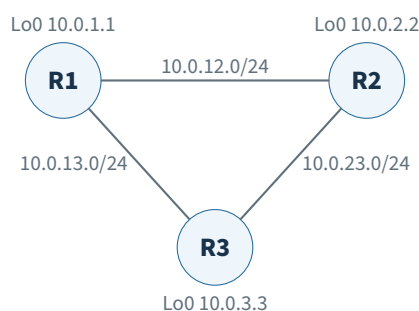
Before you start: Interface addressing, VRP navigation, and direct-link verification from Lab 2.

By the end, you can: Read a routing table, explain why remote networks initially fail, configure next-hop static routes, use route preference for a backup path, and verify reachability.

3.1 The topology: three routers and three transit networks

Three routers form a triangle, and each has a loopback network. Routers learn directly connected networks automatically; remote networks require a route.

The triangle: every router has two ways to reach the third



The third octet of each transit network names its two routers: 10.0.**12**.0 joins R1 and R2. A router's own address ends in its number, so R2 is 10.0.12.**2** and 10.0.23.**2**.

Link or loopback	Network	Example addresses in source
R1–R2	10.0.12.0/24	R1 .1, R2 .2
R1–R3	10.0.13.0/24	R1 .1, R3 .3
R2–R3	10.0.23.0/24	R2 .2, R3 .3
R1 Loopback0	10.0.1.0/24	10.0.1.1
R2 Loopback0	10.0.2.0/24	10.0.2.2
R3 Loopback0	10.0.3.0/24	10.0.3.3

Source visual — Static-routing topology from the supplied Lab 3 slide

Topology

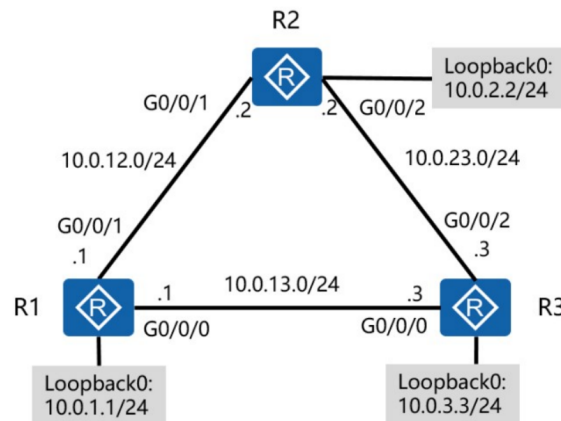


Figure 4.1 Lab topology for static and default routes

3.2 Why the first remote ping fails

After addressing, each router can reach its direct neighbors. R2 cannot reach R3's loopback 10.0.3.0/24 until its routing table identifies a path.

Source visual — R2 routing table before the missing remote routes are added

```
<R2>display ip routing-table
Route Flags: R - relay, D - download to fib
-----
Routing Tables: Public
  Destinations : 13      Routes : 13

Destination/Mask    Proto   Pre  Cost Flags NextHop         Interface
10.0.2.0/24         Direct  0    0          D  10.0.2.2 LoopBack0
10.0.2.2/32         Direct  0    0          D  127.0.0.1 LoopBack0
10.0.2.255/32       Direct  0    0          D  127.0.0.1 LoopBack0
10.0.12.0/24        Direct  0    0          D  10.0.12.2 GigabitEth
10.0.12.2/32        Direct  0    0          D  127.0.0.1 GigabitEth
10.0.12.255/32      Direct  0    0          D  127.0.0.1 GigabitEth
10.0.23.0/24        Direct  0    0          D  10.0.23.2 GigabitEth
10.0.23.2/32        Direct  0    0          D  127.0.0.1 GigabitEth
10.0.23.255/32      Direct  0    0          D  127.0.0.1 GigabitEth
127.0.0.0/8         Direct  0    0          D  127.0.0.1 InLoopBack
127.0.0.1/32        Direct  0    0          D  127.0.0.1 InLoopBack
127.255.255.255/32 Direct  0    0          D  127.0.0.1 InLoopBack
```

Key idea

A router matches the destination IP against its routing table. Without a matching route, it cannot forward the packet.

3.3 Static-route syntax in this lab

The source demonstrates next-hop static routes in the form:

```
ip route-static <destination-network> <mask/prefix> <next-hop> [preference <value>]
```

A static route says: *send this destination network to this next hop.*

3.4 Primary routes on R2

The lab directs R2 toward R3 by using next hop 10.0.23.3.

```
[R2] ip route-static 10.0.13.0 24 10.0.23.3
[R2] ip route-static 10.0.3.0 24 10.0.23.3
<R2> display ip routing-table
```

On the lab's VRP image, a static route has default preference 60, so the primary route needs no explicit value.

Worked configuration

R2 matches 10.0.3.3 to 10.0.3.0/24 and forwards it to 10.0.23.3. R3 delivers it locally; the reply still needs a return route.

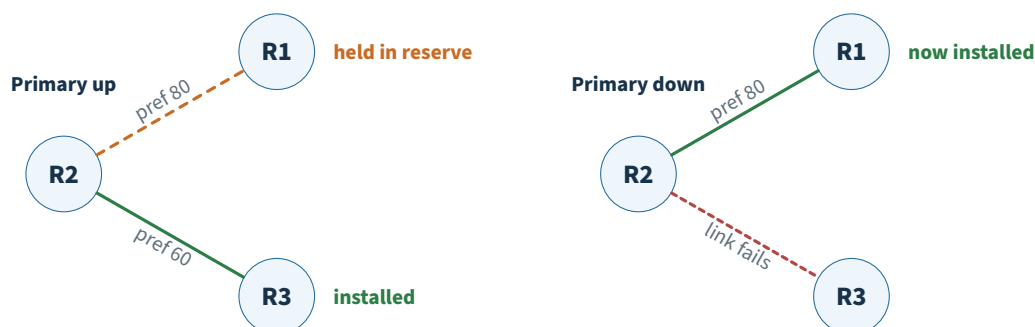
3.5 Backup static routes and route preference

The alternate path through R1 uses a higher, less-preferred value, so it remains a backup.

```
[R2] ip route-static 10.0.13.0 255.255.255.0 10.0.12.1 preference 80
[R2] ip route-static 10.0.3.0 24 10.0.12.1 preference 80
```

The route through R3 (preference 60) wins over the backup through R1 (80) until the primary path fails.

Only ONE route is installed at a time; the other waits



Lower preference wins. Both routes are configured the whole time, but the routing table shows only the better one. The backup is not a second path carrying traffic — it is a replacement that appears when the first is withdrawn.

Common mistake

Failover is not instant. The router must first decide the next hop is unreachable, and only then install the backup. Expect to lose several pings during the switch — that gap is the cost of detection, and it is why fast-converging protocols like OSPF (Chapter 4) exist.

Common mistake

One supplied slide shows 255 . 255 . 25 . 0 while explaining a /24 mask. Elsewhere the same lab consistently uses /24, whose decimal form is 255 . 255 . 255 . 0. The notes follow the lab's /24 objective and flag the source typo rather than treating it as a new subnet mask.

3.6 Verification sequence

Step 1: Use `display ip interface brief` to confirm all expected interfaces and addresses.

Step 2: Ping directly connected neighbors first; a remote-routing test is meaningless if the local link is broken.

Step 3: Use `display ip routing-table` and confirm that the intended static destination networks are present.

Step 4: Ping the remote interface or loopback address.

Step 5: For the backup-route portion, make the primary path unavailable as directed in the lab and confirm the alternate route becomes effective.

Verification evidence

Use both routing-table and ping evidence: one proves the route is installed; the other proves forwarding works.

Check your understanding

1. Why can R2 reach a directly connected network without a static route?
2. What information does the next-hop address provide?
3. Why is the backup route configured with a preference value of 80 rather than 60?
4. Which command proves that the route exists before you test it with ping?

Lab recap

Static routes map destinations to next hops. Verify direct links, add routes in both directions, inspect the table, and test traffic. A higher preference value can provide a backup path.

PRACTICAL MILESTONE

Lab 3 of 9

Static routing and the backup route

Three routers, three networks, and one deliberate failure

Coverage. Chapter 3.

Goal. Make a router forward to a network it is not attached to, then prove a backup route takes over when the primary fails.

Part A — Why the first ping fails.

Configure the interfaces, then ping from R1's LAN to R3's LAN before adding any routes. It will fail, and understanding exactly why is the point of the lab.

Key idea

A router forwards only to networks in its routing table. Directly connected networks appear automatically; everything else must be learned — statically here, dynamically in Lab 4.8. R1 has no entry for R3's LAN, so it drops the packet and reports *Destination host unreachable*. The packet never reaches R2. Nothing is broken; the router is doing exactly what it was told.

Confirm before configuring:

```
<R1> display ip routing-table
```

Count the entries. Every one should be Direct. Record that output — it is the "before" half of your evidence.

Part B — Static route syntax.

```
ip route-static <destination> <mask> <next-hop>
```

The next hop must be an address R1 can already reach — normally the neighbouring router's interface on the shared link. A next hop R1 cannot resolve produces a route that looks correct in the table and silently discards traffic.

Common mistake

The destination is a *network* address, not a host address.

`ip route-static 192.168.3.0 24 10.0.12.2` is a route to a network.

`ip route-static 192.168.3.1 24 10.0.12.2` is a typo that VRP will accept, normalise, and then behave unexpectedly.

Always write the network address with host bits zeroed.

Part C — Routes are one-directional.

Add routes on R1 so it can reach R3's LAN. Ping again. It still fails.

Key idea

This is the single most common static-routing error, and it is worth meeting deliberately. Your ping now reaches R3. R3 has no route back to R1's LAN, so the *reply* is dropped. From R1 the symptom looks identical to the original failure, which is why students often conclude their route did not work.

Connectivity requires a route in both directions. Always configure and verify both ends before concluding anything.

Prove it with `tracert` from R1: the trace advances further than before, then stops. That is the return path failing, not the forward path.

Part D — Backup routes and preference.

Add a second route to the same destination through a different neighbour, with a worse preference:

```
ip route-static 192.168.3.0 24 10.0.12.2          # primary, preference 60
ip route-static 192.168.3.0 24 10.0.13.3 preference 100 # backup
```

Key idea

Preference (Huawei's term for administrative distance) ranks *sources* of routing information. Lower wins. Static routes default to 60; OSPF internal routes are 10.

Only the preferred route is installed in the forwarding table. The backup sits inactive until the primary's next hop becomes unreachable, at which point it is installed automatically.

This is failover, not load sharing. Traffic does not split across the two paths — give them equal preference if that is what you want.

Part E — Prove the failover.

Step 1: `display ip routing-table` — confirm only the primary is present, with preference 60.

Step 2: Start a long ping from R1's LAN so you can watch the interruption.

Step 3: Shut down the primary link: `interface GigabitEthernet 0/0/0` then `shutdown`.

Step 4: `display ip routing-table` again — the backup should now be installed.

Step 5: Count how many pings were lost during the transition.

Step 6: `undo shutdown` and confirm the primary returns.

That lost-ping count is the real result of this lab. Static failover is not instant: the router must first decide the next hop is unreachable.

When it does not work

Route shows in the table but traffic fails. The next hop is probably unreachable. Ping the next hop itself from the same router.

Backup never activates. Shutting an interface on the far end may leave the local interface up, so the next hop still appears valid. Shut the local interface, or verify with `display ip routing-table protocol static`.

Both routes appear active. They have equal preference, so you have configured load sharing rather than failover.

Try it yourself

Replace both static routes with one default route, `ip route-static 0.0.0.0 0 10.0.12.2`. Does connectivity still work? What have you given up, and in what topology would that choice be dangerous?

Verification evidence

- Routing tables captured before routes, after routes, and after the failover.
- Both directions configured, with the one-way failure demonstrated rather than skipped.
- Preference values visible in the table output.
- Packet loss during failover counted and reported.

Lab deliverable

Submit configurations for all three routers, the three routing-table captures, evidence of the one-way failure and its fix, and the measured packet loss during failover with one sentence explaining why it is not zero.

Dynamic Routing with OSPF

Lab at a glance

Before you start: Static routing, interface addressing, routing-table interpretation, and ping verification.

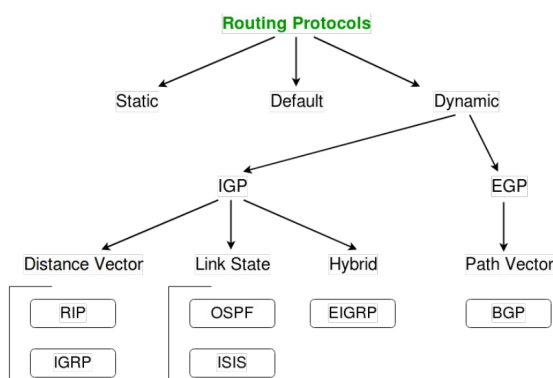
By the end, you can: Configure router IDs, advertise networks into OSPF area 0, inspect OSPF-learned routes and neighbors, and explain why dynamic routing reduces manual route maintenance.

4.1 Where OSPF fits among routing approaches

OSPF is a dynamic, interior, link-state routing protocol. Instead of configuring every destination and next hop, routers form neighbor relationships and exchange topology information.

Source visual — Routing-protocol classification shown in the supplied Lab 4 slide

Illustration of Routing Protocols in Computer Network



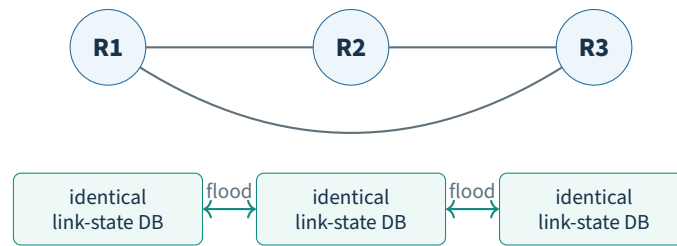
4.2 Why dynamic routing is useful

Dynamic routing reduces manual maintenance and adapts to topology changes. OSPF uses a hierarchy centered on **area 0**; this exercise places every participating interface in that single area.

4.2.1 What changes compared with Chapter 3

In the static lab you wrote a route on R1 and a mirror route on R3, then a backup pair on top, and a ping only worked once every one of them was present. Here you tell each router which of *its own* interfaces take part, and reachability follows.

Every router holds the same map, then computes its own routes



Same database everywhere. Each router then runs Dijkstra **from itself**, so the routing tables all differ — and all agree.

Key idea

A static route states a *conclusion* (“send this network that way”). OSPF distributes the *evidence* — which links exist and what they cost — and lets each router reach its own conclusion. That is why a failed link needs no new configuration: the evidence changes, and every router recomputes.

4.3 Single-area OSPF topology

R1 connects to R2 over 10.0.12.0/24 and R3 over 10.0.13.0/24. Loopbacks provide stable networks at 10.0.1.1/24, 10.0.2.2/24, and 10.0.3.3/24; the source also shows 172.16.0.1/24 on R3’s Loopback2.

Source visual — OSPF single-area topology from the supplied Lab 4 slide

ology

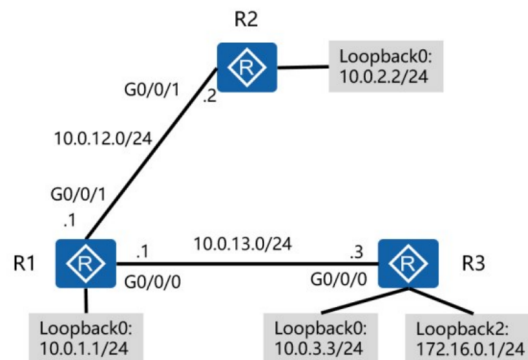


Figure 4.2 OSPF single area topology

4.4 Router IDs and OSPF process 1

Each OSPF process receives a unique router ID matching the router’s loopback address.

```
[R1] ospf 1 router-id 10.0.1.1
[R1-ospf-1] area 0
[R1-ospf-1-area-0.0.0.0] network 10.0.1.0 0.0.0.255
[R1-ospf-1-area-0.0.0.0] network 10.0.13.0 0.0.0.255
[R1-ospf-1-area-0.0.0.0] network 10.0.12.0 0.0.0.255

[R2] ospf 1 router-id 10.0.2.2
[R2-ospf-1] area 0
[R2-ospf-1-area-0.0.0.0] network 10.0.2.0 0.0.0.255
[R2-ospf-1-area-0.0.0.0] network 10.0.12.0 0.0.0.255
```

R3 follows the same pattern for the networks connected to it.

Key idea

The `network` statements select local interfaces for OSPF. An unmatched interface does not participate or advertise its connected network through that process.

4.5 Understand the wildcard mask used in the slides

The wildcard `0.0.0.255` matches addresses in the corresponding `x.x.x.0/24` network.

4.6 Verify convergence

After configuration, do not jump directly to ping. First verify the control plane.

```
<R1> display ip routing-table
<R1> display ospf peer
<R1> display ospf peer brief
```

The detailed output includes neighbor ID, address, state, priority, DR/BDR data, timers, and uptime; the brief form shows only the essentials.

Source visual — Condensed OSPF peer information from the supplied Lab 4 material

and can

d version

OSPF Process 1 with Router ID 10.0.1.1

Peer Statistic Information

Area Id	Interface	Neighbor id	St
0.0.0.0	GigabitEthernet0/0/0	10.0.3.3	Full
0.0.0.0	GigabitEthernet0/0/1	10.0.2.2	Full

<R2> display ospf peer brief

OSPF Process 1 with Router ID 10.0.2.2

Peer Statistic Information

Area Id	Interface	Neighbor id	State
0.0.0.0	GigabitEthernet0/0/1	10.0.1.1	Full

<R3> display ospf peer brief

OSPF Process 1 with Router ID 10.0.3.3

Peer Statistic Information

Area Id	Interface	Neighbor id	State
0.0.0.0	GigabitEthernet0/0/0	10.0.1.1	Full

Verification evidence

Require three checks: the expected neighbor is **Full**, remote networks appear as OSPF routes, and ping succeeds.

4.7 Connectivity test

The lab pings from R2 to R1 and R3's loopback 10.0.3.3, confirming that OSPF learned the remote paths.

4.8 Troubleshooting OSPF in the order the evidence suggests

Step 1: Confirm interface IP addresses and physical/protocol state.

Step 2: Confirm each required network is matched by the OSPF configuration in area 0.

Step 3: Use `display ospf peer brief`. If the expected neighbor is absent or not Full, solve the adjacency problem before studying remote routes.

Step 4: Use `display ip routing-table` and locate the OSPF-learned destination.

Step 5: Only then use ping to verify the data path.

Common mistake

A missing route may reflect a failed adjacency. Do not mask the fault with a static route; inspect the interface and OSPF configuration.

Check your understanding

1. Why does the lab explicitly configure a router ID?
2. What is the role of area 0 in the source material?
3. Which command gives the compact neighbor summary?
4. What two pieces of evidence should be present before you trust a successful OSPF configuration?

Lab recap

OSPF exchanges topology information through neighbor relationships. Configure unique router IDs and area networks, verify Full peers and learned routes, then test connectivity.

PRACTICAL MILESTONE

Lab 4 of 9

OSPF: let the network work it out

Replace your static routes with a protocol that adapts

Coverage. Chapter 4.

Goal. Configure single-area OSPF, watch adjacencies form, and see the network reroute itself without you.

How to use this lab

Use the same topology as Lab 3.6. Remove every static route first — leaving them in place means you will not know which mechanism is carrying your traffic, and static routes have a better preference so they will win.

```
undo ip route-static 192.168.3.0 24 10.0.12.2
```

Part A — Why bother.

Static routing worked in Lab 3.6. It also required you to configure every route on every router in both directions, and to predict every failure in advance. Add a fourth router and the work grows faster than the network.

Key idea

Static routing scales with the number of *paths*. Dynamic routing scales with the number of *routers*.

OSPF routers describe their own links to each other, and every router independently computes shortest paths across the resulting map. Add a link and it is advertised automatically; lose a link and the recalculation happens without you.

Part B — Router IDs.

```
system-view
ospf 1 router-id 1.1.1.1
 area 0
   network 10.0.12.0 0.0.0.255
   network 192.168.1.0 0.0.0.255
quit
quit
```

Common mistake

Set the router ID explicitly. If you do not, VRP picks one from the interface addresses — and it may pick a different one after a reboot, which tears down every adjacency and produces a fault that looks random.

A router ID is a 32-bit identifier that merely looks like an IP address. It does not need to be reachable, and by convention it is 1.1.1.1, 2.2.2.2, and so on.

Part C — The wildcard mask.

OSPF's `network` statement takes a wildcard mask, which is the bitwise inverse of a subnet mask. This trips up nearly everyone the first time.

Prefix	Subnet mask	Wildcard	Matches
/24	255.255.255.0	0.0.0.255	256 addresses
/30	255.255.255.252	0.0.0.3	4 addresses
/32	255.255.255.255	0.0.0.0	exactly one
/16	255.255.0.0	0.0.255.255	65 536 addresses

The rule: subtract each octet of the subnet mask from 255. The supplied `wildcard_calc.py` does this and shows the working, but do a few by hand first — it appears in examinations.

Key idea

A 0 bit in a wildcard means “this bit must match”; a 1 bit means “do not care”. So 0.0.0.255 means the first three octets must match and the last is free — exactly a /24.

The `network` statement does not advertise a network. It selects which *interfaces* participate in OSPF: any interface whose address falls inside the range is enabled.

Part D — Watch the adjacency form.

```
<R1> display ospf peer brief
Area Id      Interface      Neighbor id    State
0.0.0.0      GigabitEthernet0/0/0  2.2.2.2      Full
```

Key idea

Full is the only state that means success. Anything else is a diagnosis:

Init — hellos are being sent but the neighbour is not acknowledging them. Usually one-way connectivity.

2-Way — neighbours see each other; on a broadcast segment, non-designated routers legitimately stay here.

ExStart / Exchange — stuck here almost always means an MTU mismatch across the link.

Full — databases synchronised, routes exchanged.

Neighbours also fail to form if hello and dead intervals differ, if area IDs differ, or if authentication is configured on one side only.

Part E — Read the routing table again.

```
<R1> display ip routing-table protocol ospf
```

OSPF routes carry preference 10 against static routing's 60, so OSPF wins wherever both exist. Note the Cost column: Huawei derives it from interface bandwidth, so a gigabit path is preferred over a serial one without you saying so.

Part F — Break it, and time the recovery.

Start a continuous ping, shut down the primary link, and count lost packets — then compare with the figure from Lab 3.6.

When it does not work

Neighbour never leaves Init. Traffic flows one way only. Check masks on both ends and any access lists.

Stuck in ExStart or Exchange. MTU mismatch. Compare `display interface` on both sides.

Adjacency forms but no routes appear. The `network` statement covers the link but not the LAN interface. Add a statement covering it.

Routes appear then vanish repeatedly. Duplicate router IDs. Every router in the area must be unique.

Try it yourself

Compare your two failover measurements. OSPF's default dead interval is 40 seconds, so recovery is not instant — but it required no backup route from you, and it would have handled a failure you never anticipated.

State the trade-off in one sentence, then say which you would deploy on a three-router branch network and why.

Verification evidence

- All static routes removed before OSPF is configured.
- Router IDs set explicitly and unique.
- `display ospf peer brief` shows Full on every adjacency.
- OSPF routes present with preference 10 and their costs.
- Failover time measured and compared against the static-routing result.

Lab deliverable

Submit configurations, peer-brief output showing Full adjacencies, the OSPF routing table, your wildcard-mask working for each network statement, and a comparison of static versus OSPF failover with measured packet loss for both.

FTP Service and File Transfer

Lab at a glance

Before you start: Direct IPv4 reachability between two routers and basic VRP user/configuration views.

By the end, you can: Enable an FTP server, configure an authorized FTP user, connect from a client router, list files, transfer files in binary mode, verify transfers, close the session, and clean up safely.

5.1 Service model

FTP transfers files between a client and server. Here R1 is the server, R2 the client, and both share 10.0.12.0/24.

Source visual — R1-R2 FTP topology from the supplied Lab 5 slide

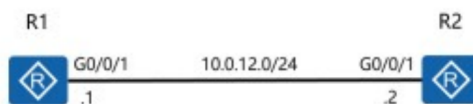


Figure 5.1 FTP topology

Device	Role	Interface	Address
R1	FTP server	G0/0/1	10.0.12.1/24
R2	FTP client	G0/0/1	10.0.12.2/24

5.2 Step 1: prove IP connectivity before touching FTP

```
[R1] interface GigabitEthernet 0/0/1
[R1-GigabitEthernet0/0/1] ip address 10.0.12.1 24
[R2] interface GigabitEthernet 0/0/1
[R2-GigabitEthernet0/0/1] ip address 10.0.12.2 24
<R1> ping 10.0.12.2
```

Key idea

Verify IP reachability before troubleshooting FTP.

5.3 Step 2: enable the FTP server and define its working directory

Enable the server on R1 and set its default directory to `flash:/`.

```
[R1] ftp server enable
[R1] set default ftp-directory flash:/
```

5.4 Step 3: authorize an FTP user

Under AAA, create user `huawei`, set its password, service type, privilege, and FTP directory.

```
[R1] aaa
[R1-aaa] local-user huawei password cipher huawei123
[R1-aaa] local-user huawei service-type ftp
[R1-aaa] local-user huawei privilege level 15
[R1-aaa] local-user huawei ftp-directory flash:/
```

5.5 Step 4: verify the server process

Use `display ftp-server` to confirm that the service is running on TCP port 21.

Source visual — FTP server status output shown in the source material

```
[R1]display ftp-server

FTP server is running

Max user number          5
User count                0
Timeout value(in minute) 30
Listening port           21
Acl number                0
FTP server's source address 0.0.0.0
```

Verification evidence

Before login, prove that IP works, the server is running, and the user has the correct service type and directory.

5.6 Step 5: log in from R2


```
<R2> ftp 10.0.12.1
User(10.0.12.1:(none)): huawei
Enter password: *****
[R2-ftp]
```

Source visual — Successful FTP client login from R2

```
<R2>ftp 10.0.12.1
Trying 10.0.12.1 ...
Press CTRL+K to abort
Connected to 10.0.12.1.
220 FTP service ready.
User(10.0.12.1:(none)):huawei
331 Password required for huawei.
Enter password:
230 User logged in.

[R2-ftp]
```

The [R2-ftp] prompt confirms that the device is now in the FTP client session view.

5.7 Step 6: inspect, download, and upload files

Use `dir` before and after transfers, and select binary mode for the configuration archive.

```
[R2-ftp] dir
[R2-ftp] binary
[R2-ftp] get vrpcfg.zip vrpnew.zip
[R2-ftp] bye
```

If `vrpcfg.zip` is absent on R1, run `save` to create it.

To upload a file:

```
[R2-ftp] put vrpnew.zip vrpnew2.zip
```

Then check the server directory.

Concept map

dir = inspect remote files | **get** = server → client | **put** = client → server | **bye** = close the FTP session.

5.8 Step 7: clean up carefully

Remove only the test files created by the exercise.

```
<R1> delete flash:/vrpnew2.zip
<R2> delete flash:/vrpnew.zip
```

Common mistake

Deletion is destructive. Verify the full path and filename; never erase the entire `flash:/` directory.

5.9 A practical troubleshooting order

Step 1: Ping the server IP from the client.

Step 2: Run `display ftp-server` on R1 and confirm the service is running.

Step 3: Check the local user's password, service type, privilege, and FTP directory.

Step 4: Attempt login and distinguish a connectivity failure from an authentication failure.

Step 5: Use `dir` to verify the expected file and directory before transfer.

Step 6: After `get` or `put`, inspect the destination directory to prove the file arrived.

Check your understanding

1. Why does the lab ping R2 before configuring the FTP application?
2. Which command proves that the FTP server is running?
3. What is the direction of a `get` operation?
4. Why does the lab use `dir` both before and after transfers?

Lab recap

FTP depends on working IP connectivity. AAA authorizes access; `dir`, `get`, and `put` inspect and transfer files; final directory checks prove success.

PRACTICAL MILESTONE

Lab 5 of 9

FTP: a service on top of a working network

Turn a router into a file server, in the right order

Coverage. Chapter 5.

Goal. Configure an FTP server and client, and internalise the habit of proving each layer before building on it.

How to use this lab

This lab is short, and its real lesson is diagnostic order. Every step below verifies the layer beneath it before adding anything. Skip a verification and a later failure becomes ambiguous — you will not know whether the problem is routing, authentication, or the file system.

Step 1 — Prove IP connectivity first.

Before touching FTP, ping the server from the client.

Key idea

FTP is an application-layer service riding on TCP, riding on IP. If IP does not work, FTP cannot, and every FTP symptom you see will be misleading.

Ping first. Always. It costs five seconds and eliminates three layers of possible causes.

Step 2 — Enable the server and set its directory.

```
system-view
ftp server enable
```

Confirm the file system has something to serve:

```
<R1> dir flash:
```

Step 3 — Authorise a user.

```
aaa
local-user ftpuser password cipher Huawei@123
local-user ftpuser privilege level 15
local-user ftpuser service-type ftp
local-user ftpuser ftp-directory flash:
quit
```

Common mistake

All four `local-user` lines are required, and omitting any one produces a different, confusing failure:

No `service-type ftp` — login is refused even with the correct password.

No `ftp-directory` — login succeeds, then `dir` returns nothing or errors.

Insufficient `privilege level` — read works, upload is denied.

Wrong password — the only failure that says what it means.

If login behaves oddly, re-read this block before suspecting the network.

Step 4 — Verify the server is listening.

```
<R1> display ftp-server
```

Step 5 — Connect from the client.

```
<R2> ftp 192.168.1.1
Trying 192.168.1.1 ...
Press CTRL+K to abort
Connected to 192.168.1.1.
220 FTP service ready.
User(192.168.1.1:(none)):ftpuuser
331 Password required for ftpuuser.
Enter password:
230 User logged in.
[R2-ftp]
```

The numeric codes are the protocol telling you where you are. Learn the four that matter:

Code	Meaning
220	Server ready — TCP reached the FTP service
331	Username accepted, password required
230	Authenticated
530	Login failed — credentials or service-type
550	Action not taken — permissions or missing file

Key idea

Reaching 220 means IP, TCP, and the FTP service are all working. A failure after 220 is authentication or file permissions — never routing.
That single observation splits your troubleshooting in half.

Step 6 — Transfer in both directions.

```
[R2-ftp] dir
[R2-ftp] binary
[R2-ftp] get vrpcfg.zip
[R2-ftp] put test.txt
[R2-ftp] quit
```

Common mistake

Set binary before transferring anything that is not plain text.
ASCII mode rewrites line endings, which silently corrupts compressed files, images, and firmware. The transfer reports success and the file is unusable — one of the more frustrating faults to diagnose, because nothing announces it.

When it does not work

Connection refused. `ftp server enable` not applied, or you are pinging a different interface than the one you connected to.

530 login failed. Missing `service-type ftp`, or the password was entered in plaintext where cipher was configured.

Logged in but dir is empty. `ftp-directory` not set, or set to a path that does not exist.
`put` returns **550**. Privilege level too low, or the flash file system is full. Check with `dir flash:`.

Try it yourself

Capture the session in Wireshark using the skills from Lab 1.5, filtered with `ftp`. Find your password in the capture. It is there in plaintext, because FTP does not encrypt the control channel. Then state in one sentence why FTP should not cross an untrusted network, and name the protocol you would use instead.

Verification evidence

- Ping succeeded before any FTP configuration was attempted.
- `display ftp-server` confirms the service is enabled.
- Session transcript includes the 220, 331, and 230 codes.
- Both `get` and `put` demonstrated, with `binary` set.
- Wireshark capture shows the credentials in plaintext.

Lab deliverable

Submit the server configuration, `display ftp-server` output, a full session transcript with response codes, evidence of a successful transfer in each direction, and the Wireshark screenshot showing the plaintext password with a one-paragraph security comment.

DHCP Address Allocation

Lab at a glance

Before you start: Router and switch interface configuration, IPv4 subnetting, and basic reachability.

By the end, you can: Explain DHCP's purpose, enable DHCP, build global and interface-based pools, configure switches to request addresses, reserve/exclude addresses, set leases and DNS information, and verify allocation state.

6.1 What DHCP automates

Dynamic Host Configuration Protocol (DHCP) assigns clients an IP address and options such as a subnet mask, gateway, and DNS server. It improves consistency and reduces manual administration.

6.1.1 The four-message exchange

A client with no address cannot yet be addressed, so the conversation begins as a broadcast. The four steps are remembered as **DORA**.

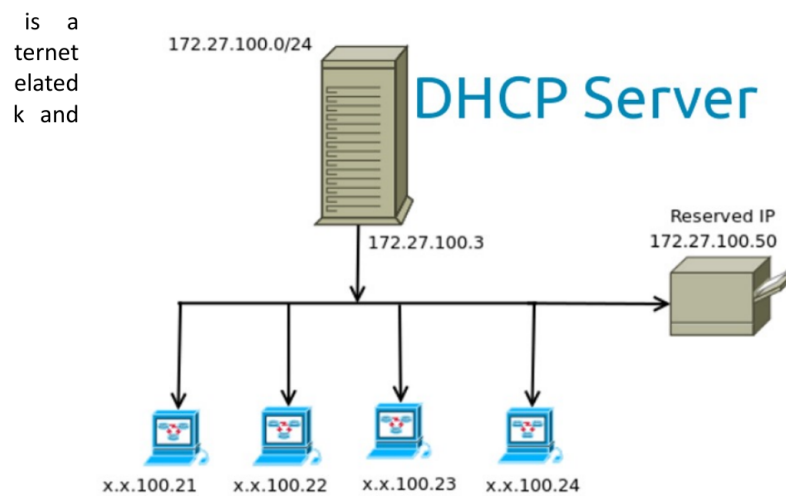
DORA — how a client with no address gets one



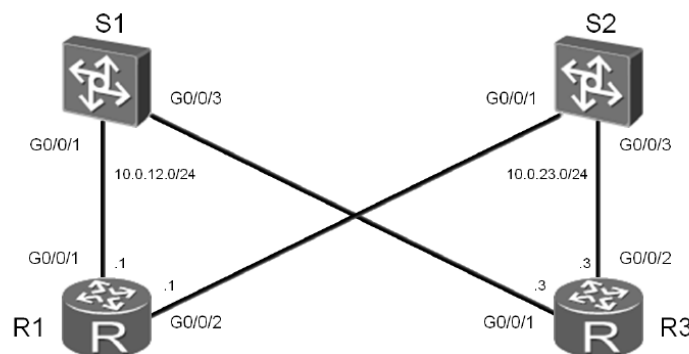
The **Request** is broadcast, not sent directly to the chosen server. That is deliberate: if two servers made offers, the broadcast tells the one that was *not* chosen to release the address it had reserved.

Key idea

Discover and Request are broadcasts. Routers do not forward broadcasts, so a client and its DHCP server must share a broadcast domain — or something must carry the request across. That “something” is the relay, in Section 6.9.

Source visual — DHCP server-and-clients illustration from the supplied overview**6.2 The detailed lab topology**

R1 and R3 act as DHCP servers; S1 and S2 are clients. The lab compares named global pools with interface-based pools.

Source visual — DHCP topology used in the detailed supplied lab

Mechanism	How the source configures it
Global pool	Define a named ip pool, then select it with <code>dhcp select global</code> .
Interface-based pool	Use <code>dhcp select interface</code> ; configure exclusions, DNS, and lease settings under that interface.

6.3 Step 1: prepare interfaces and isolate the intended path

Configure addressing and disable unused paths before testing, so only the intended DHCP server can respond.

6.4 Step 2: enable DHCP globally on the routers

Enable DHCP globally before configuring pools.

```
[R1] dhcp enable
[R3] dhcp enable
```

6.5 Step 3: create the global address pools

R1 creates pool1 for 10.0.12.0/24; R3 creates pool2 for 10.0.23.0/24. Each lease lasts 1 day and 12 hours.

```
[R1] ip pool pool1
[R1-ip-pool-pool1] network 10.0.12.0 mask 24
[R1-ip-pool-pool1] gateway-list 10.0.12.1
[R1-ip-pool-pool1] lease day 1 hour 12
[R1] interface GigabitEthernet 0/0/1
[R1-GigabitEthernet0/0/1] dhcp select global

[R3] ip pool pool2
[R3-ip-pool-pool2] network 10.0.23.0 mask 24
[R3-ip-pool-pool2] gateway-list 10.0.23.3
[R3-ip-pool-pool2] lease day 1 hour 12
[R3] interface GigabitEthernet 0/0/2
[R3-GigabitEthernet0/0/2] dhcp select global
```

6.6 Step 4: make the switches DHCP clients

Configure the management interface as a DHCP client.

```
[S1] dhcp enable
[S1] interface Vlanif 1
[S1-Vlanif1] ip address dhcp-alloc
<S1> display ip interface brief
```

Repeat on S2, then inspect the corresponding server pool.

Command map

Command	What it proves
display ip pool name pool1	R1 pool settings and allocation counters.
display ip pool name pool2	R3 pool settings and allocation counters.
display ip interface brief	Address and state of the switch Vlanif1.

6.7 Step 5: build interface-based address pools

Next, configure DHCP directly under the active routed interfaces.

```
[R1] interface GigabitEthernet 0/0/2
[R1-GigabitEthernet0/0/2] dhcp select interface
[R1-GigabitEthernet0/0/2] dhcp server dns-list 10.0.23.254
[R1-GigabitEthernet0/0/2] dhcp server excluded-ip-address 10.0.23.254
[R1-GigabitEthernet0/0/2] dhcp server lease day 1 hour 12

[R3] interface GigabitEthernet 0/0/1
[R3-GigabitEthernet0/0/1] dhcp select interface
[R3-GigabitEthernet0/0/1] dhcp server dns-list 10.0.12.254
[R3-GigabitEthernet0/0/1] dhcp server excluded-ip-address 10.0.12.254
[R3-GigabitEthernet0/0/1] dhcp server lease day 1 hour 12
```


Key idea

excluded-ip-address prevents the DNS server address from being leased to a client.

6.8 Step 6: force a fresh client request

Cycle Vlanif1 to force a fresh request from the interface-based pool.

```
[S2] interface Vlanif 1
[S2-Vlanif1] shutdown
[S2-Vlanif1] undo shutdown

[R1] interface GigabitEthernet 0/0/2
[R1-GigabitEthernet0/0/2] undo shutdown
```

Repeat for S1 and R3 on the other subnet.

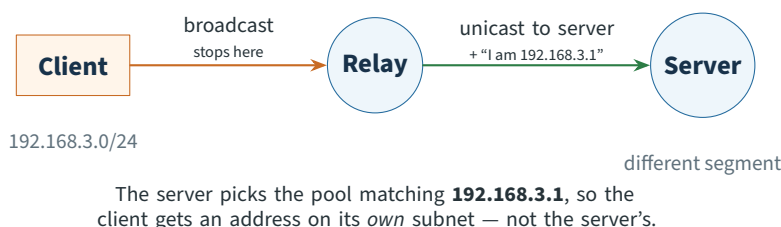
Verification evidence

Success requires a client address from the expected subnet and a matching used lease in the server pool.

6.9 DHCP relay: reaching a server on another segment

Everything above assumes the client and the server share a segment. Real networks centralise the DHCP server, so the client's broadcast has to cross a router that, by design, will not forward it.

A **relay agent** solves this by translating the conversation. The router receives the broadcast on the client side, rewrites it as a *unicast* to the configured server, and inserts the receiving interface's address so the server knows which pool to allocate from.

The relay converts a broadcast into a unicast, and back

```
[R1] dhcp enable
[R1] interface GigabitEthernet 0/0/2
[R1-GigabitEthernet0/0/2] ip address 192.168.3.1 24
[R1-GigabitEthernet0/0/2] dhcp select relay
[R1-GigabitEthernet0/0/2] dhcp relay server-ip 10.0.12.1
```

Common mistake

The relay must still be able to reach the server by ordinary routing, and the server needs a *return* route to the client subnet. A relay that is configured correctly but has no return path produces the same silent symptom as no relay at all: the client waits and eventually gives up.

6.10 Troubleshooting DHCP

Step 1: Confirm the server-side routed interface is up and has the intended subnet address.

Step 2: Confirm `dhcp enable` is present on the router.

Step 3: For a global pool, verify the pool network/gateway and `dhcp select global` on the correct interface.

Step 4: For an interface pool, verify `dhcp select interface` plus any excluded/DNS/lease settings.

Step 5: Confirm the client has DHCP enabled and uses `ip address dhcp-alloc` on Vlanif1.

Step 6: Cycle the client interface if the exercise requires a new lease, then inspect the client and pool state again.

Check your understanding

1. What is the difference between a named global pool and an interface-based pool in this lab?
2. Why does the source exclude 10.0.23.254 and 10.0.12.254 from allocation?
3. Which command proves that a pool has allocated an address?
4. Why does the lab shut down unused links during preparation?

Lab recap

DHCP automates address allocation through global or interface pools. Verify both sides: the client has the expected address and the server records the lease.

PRACTICAL MILESTONE

Lab 6 of 9

DHCP: addresses without a spreadsheet

Global pools, interface pools, and the relay that crosses a router

Coverage. Chapter 6.

Goal. Hand out addresses automatically, understand why the process works at all before a client has an address, and extend it across a router.

Part A — The four-message exchange.

A client with no address has to ask for one — but it cannot address a request to a server it does not know about.

Message	Sent to	Meaning
DISCOVER	broadcast	Is there a DHCP server out there?
OFFER	broadcast	Yes, and here is an address you may have
REQUEST	broadcast	I accept that one
ACK	broadcast	Confirmed, it is yours for the lease duration

Key idea

DISCOVER is broadcast because the client has no address and no server to aim at. It fills the source field with 0.0.0.0 and the destination with 255.255.255.255.

REQUEST is also broadcast, and this is the part students find odd. The client already knows which server it wants — but if two servers made offers, the broadcast tells the *declined* server to release the address it had reserved. Unicasting the request would leave that address stranded until its offer timed out.

Part B — Global pools.

```

system-view
dhcp enable

ip pool LAN1
 network 192.168.1.0 mask 255.255.255.0
 gateway-list 192.168.1.1
 dns-list 8.8.8.8 114.114.114.114
 excluded-ip-address 192.168.1.2 192.168.1.10
 lease day 1
quit

interface GigabitEthernet 0/0/0
 dhcp select global
quit

```

Common mistake

`dhcp enable` is required globally before any pool works, and forgetting it is the most common failure in this lab. Pools configure without complaint and simply never respond.

`excluded-ip-address` matters too. Without it the pool will happily hand a client the address you assigned statically to the router itself, producing a duplicate-address conflict that appears intermittently and is unpleasant to diagnose.

Part C — Interface pools.

A global pool is defined centrally and selected per interface. An interface pool is defined on the interface itself and derives its network from that interface's address:

```
interface GigabitEthernet 0/0/1
 ip address 192.168.2.1 24
 dhcp select interface
 dhcp server dns-list 8.8.8.8
 dhcp server excluded-ip-address 192.168.2.2 192.168.2.5
quit
```

Use interface pools for a single directly attached segment; use global pools when you need one definition serving several interfaces, or when a relay is involved.

Part D — Crossing a router.

Routers do not forward broadcasts, so a client on one segment cannot reach a DHCP server on another. That is deliberate — and the reason a relay exists.

```
interface GigabitEthernet 0/0/2
 dhcp select relay
 dhcp relay server-ip 192.168.1.1
quit
```

Key idea

The relay receives the broadcast DISCOVER, converts it to a unicast addressed to the server, and — critically — records the receiving interface's address in the packet's `giaddr` field. That field is how the server knows which pool to draw from. Without it the server would see a request arriving from a router and have no idea which subnet the client is actually on.

Part E — Verify from both ends.**Command map**

Command	Shows
<code>display ip pool</code>	Pools, sizes, and how many addresses are in use
<code>display ip pool name LAN1 used</code>	Which addresses are leased, and to whom
<code>display dhcp relay all</code>	Relay configuration and target server
<code>display ip interface brief</code>	Whether the client actually received an address

When it does not work

Client gets no address. Check `dhcp enable` first — then that the interface has `dhcp select` applied at all.

Client gets an address from the wrong pool. On a relayed request, the `giaddr` does not match any pool network. Confirm the relay interface's address falls inside the intended pool.

Address conflicts appear. Static assignments are inside the pool range and not excluded.

Works on the local segment, fails across the router. Relay not configured, or pointing at an address the relay itself cannot reach. Ping the server from the relay router.

Try it yourself

Capture the exchange in Wireshark with the display filter `bootp` (Wireshark still uses DHCP's original protocol name).

Find the DISCOVER and read its source address. Then explain, in one sentence, how the server's OFFER reaches a client that does not yet have an address to send it to.

Verification evidence

- `dhcp enable` present before any pool configuration.
- Both a global pool and an interface pool demonstrated.
- Static addresses excluded from every pool range.
- `display ip pool` shows leases actually issued.
- Relay demonstrated with a client on a different segment from the server.

Lab deliverable

Submit configurations for server and relay, `display ip pool` output showing issued leases, evidence a client on a remote segment received a correct address and gateway, and a Wireshark capture of all four messages with the DISCOVER's source address highlighted.

Ethernet Link Aggregation and LACP

Lab at a glance

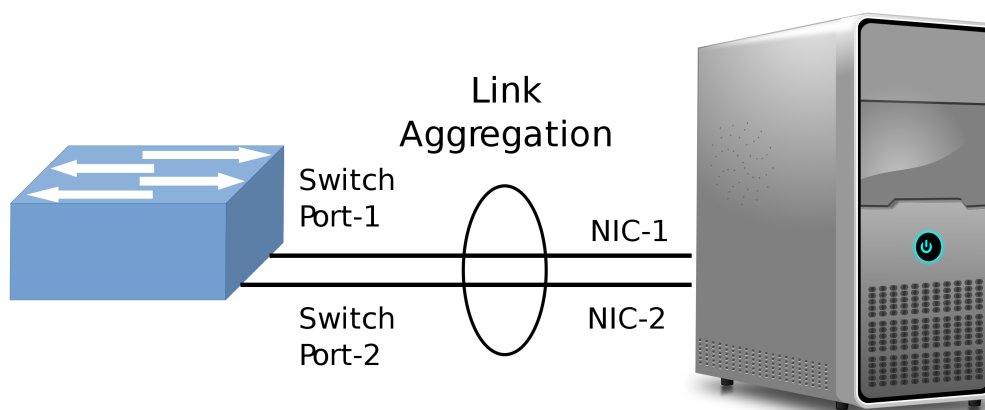
Before you start: Switch CLI navigation and Ethernet interface configuration.

By the end, you can: Set interface speed manually, create an Eth-Trunk, compare manual aggregation with static LACP, explain system/interface priority, and verify active member links.

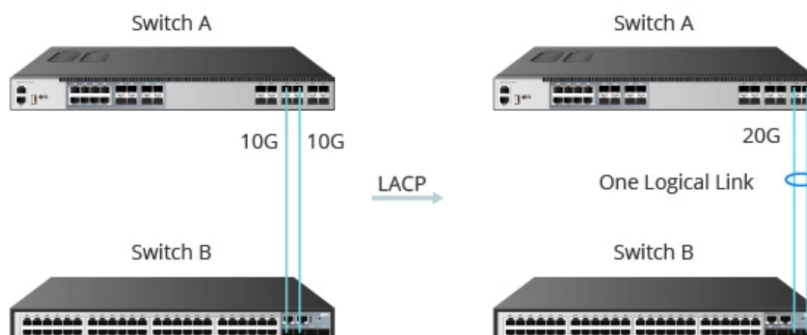
7.1 Why aggregate Ethernet links?

Link aggregation combines physical Ethernet links into one logical connection. LACP coordinates membership and reacts to member failure or recovery.

Source visual — Link aggregation between a switch and a server



Source visual — Two physical links presented as one logical link between switches



Key idea

An Eth-Trunk is the logical bundle; its physical interfaces are members. LACP coordinates the active set between peers.

7.2 Priority concepts used by the source

The lab uses two priorities; for both, **smaller values have higher priority**.

Priority	Role in the lab
LACP system priority	Helps select the device that controls active-link selection.
LACP interface priority	Ranks candidate member interfaces.

7.3 Lab topology

S1 and S2 are connected using two member links, G0/0/9 and G0/0/10. The bundle is represented as Eth-Trunk 1.

Source visual — Ethernet link-aggregation topology from the supplied Lab 7 slide

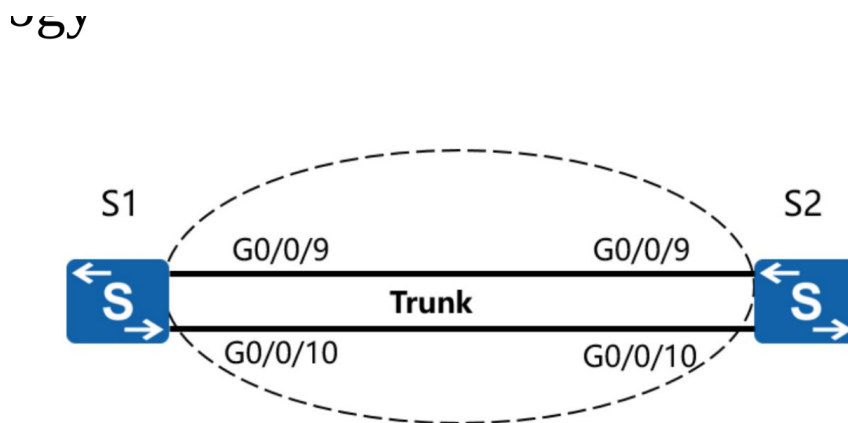


Figure 1.1 Ethernet link aggregation topology

7.4 Prepare the physical interfaces

The exercise disables auto-negotiation and sets both member links to 100 Mbit/s on each switch.

```
[S1] interface GigabitEthernet 0/0/9
[S1-GigabitEthernet0/0/9] undo negotiation auto
[S1-GigabitEthernet0/0/9] speed 100
[S1] interface GigabitEthernet 0/0/10
[S1-GigabitEthernet0/0/10] undo negotiation auto
[S1-GigabitEthernet0/0/10] speed 100
```

Configure S2 identically, then verify both interfaces with `display interface`.

7.5 Manual link aggregation

First create Eth-Trunk 1 and add both interfaces manually.

```
[S1] interface Eth-Trunk 1
[S1-Eth-Trunk1] quit
[S1] interface GigabitEthernet 0/0/9
[S1-GigabitEthernet0/0/9] eth-trunk 1
[S1] interface GigabitEthernet 0/0/10
[S1-GigabitEthernet0/0/10] eth-trunk 1
```

Repeat on S2.

7.6 Rebuild the bundle in static LACP mode

Remove the manual memberships with `undo eth-trunk`, then rebuild the trunk in LACP mode.

```
[S1] interface Eth-Trunk 1
[S1-Eth-Trunk1] mode lacp
[S1] interface GigabitEthernet 0/0/9
[S1-GigabitEthernet0/0/9] eth-trunk 1
[S1] interface GigabitEthernet 0/0/10
[S1-GigabitEthernet0/0/10] eth-trunk 1
```

Repeat on S2.

7.7 Manipulate LACP priorities

Set S1's system and member-interface priorities to 100.

```
[S1] lacp priority 100
[S1] interface GigabitEthernet 0/0/9
[S1-GigabitEthernet0/0/9] lacp priority 100
[S1] interface GigabitEthernet 0/0/10
[S1-GigabitEthernet0/0/10] lacp priority 100
```

Concept map

Physical preparation → compatible member links
↓
Eth-Trunk membership → one logical bundle
↓
Static LACP mode → protocol-coordinated active members
↓
Priority settings → influence selection behavior.

7.8 Verify the Eth-Trunk

Eth-Trunk output confirms the mode, member state, and actor/partner information.

Source visual — Eth-Trunk/LACP status output taken from the supplied Lab 7 material

```
[S1]display eth-trunk 1
Eth-Trunk1's state information is:
Local:
LAG ID: 1                WorkingMode: LACP
Preempt Delay: Disabled   Hash arithmetic: According to SIP-XOR-DIP
System Priority: 100       System ID: d0d0-4ba6-aab0
Least Active-linknumber: 1 Max Active-linknumber: 8
Operate status: up        Number Of Up Port In Trunk: 2
-----
ActorPortName      Status  PortType PortPri PortNo PortKey PortState Weight
GigabitEthernet0/0/9 Selected 100M    100     1    289   10111100 1
GigabitEthernet0/0/10 Selected 100M    100     2    289   10111100 1

Partner:
-----
ActorPortName      SysPri  SystemID      PortPri PortNo PortKey PortState
GigabitEthernet0/0/9 32768   d0d0-4ba6-ac20 32768   1    289   10111100
GigabitEthernet0/0/10 32768   d0d0-4ba6-ac20 32768   2    289   10111100

[S2]display eth-trunk 1
Eth-Trunk1's state information is:
Local:
LAG ID: 1                WorkingMode: LACP
Preempt Delay: Disabled   Hash arithmetic: According to SIP-XOR-DIP
```

Verification evidence

Confirm that Eth-Trunk 1 exists on both switches, uses LACP, contains G0/0/9 and G0/0/10, and shows the expected selected members.

7.9 Troubleshooting order

Step 1: Confirm the physical interfaces are up and use compatible speed/negotiation settings.

Step 2: Confirm the same member links are assigned to Eth-Trunk 1 on both switches.

Step 3: Confirm both sides use the same aggregation mode.

Step 4: Inspect LACP system/interface priorities if active-link selection is not what the lab expects.

Step 5: Use Eth-Trunk display output rather than checking only individual interfaces.

Common mistake

A physical link may be up but excluded from the bundle when membership or LACP mode differs across peers. Verify the logical trunk on both ends.

Check your understanding

1. What is the difference between a physical member interface and an Eth-Trunk?
2. Why does the source disable auto-negotiation before forcing 100 Mbit/s?
3. Which numerical priority is higher: 100 or 32768, according to the lab's rule?
4. What extra capability does static LACP add compared with manual aggregation?

Lab recap

Build the two-link Eth-Trunk first manually, then with LACP. Verify the logical mode and member state as well as physical links.

PRACTICAL MILESTONE

Lab 7 of 9

Link aggregation: several cables, one link

Bundle physical links, then make the bundle negotiate

Coverage. Chapter 7.

Goal. Build an Eth-Trunk in manual mode, rebuild it in LACP mode, and understand why the second is safer.

Part A — Two problems, one solution.

Aggregation solves bandwidth and redundancy together. Four gigabit links behave as one four-gigabit link, and losing one degrades capacity instead of connectivity.

Key idea

Aggregation also solves a problem you would otherwise create. Connect two switches with four parallel cables and no aggregation, and you have built four loops — spanning tree will block three of them, leaving you exactly where you started with three cables wasted.

An Eth-Trunk presents the bundle to spanning tree as a single logical interface, so all four links forward.

Part B — Traffic does not split evenly.

Common mistake

A four-link trunk does not give one flow four times the bandwidth.

The switch hashes header fields — source and destination MAC, IP, or port depending on configuration — and every frame in a given conversation lands on the same physical link. This is deliberate: splitting a flow across links would reorder packets and hurt TCP badly.

So a single large file transfer uses one link. Aggregation increases *aggregate* capacity across many flows, not the speed of any one of them. Expect uneven utilisation, and choose a hash that matches your traffic — MAC-based hashing is useless behind a router, where every frame carries the same router MAC.

Part C — Manual mode.

```
system-view
interface Eth-Trunk 1
 mode manual load-balance
quit

interface GigabitEthernet 0/0/1
 eth-trunk 1
quit
interface GigabitEthernet 0/0/2
 eth-trunk 1
quit
```

Configure the matching bundle on the far switch, then verify:

```
<SW1> display eth-trunk 1
```

Part D — Why manual mode is dangerous.

Manual mode forwards on every member port unconditionally. It never checks that the far end agrees.

Key idea

Miscable one member — into the wrong switch, or the wrong port — and manual mode forwards over it anyway. The result is a forwarding loop that spanning tree cannot see, because the trunk hides its members from STP.

That is a broadcast storm, and it takes the segment down. It is the single best argument for LACP.

Part E — Static LACP mode.

```
interface Eth-Trunk 1
  undo mode manual load-balance
  mode lacp-static
quit
```

Now each end sends LACP protocol frames announcing its system ID, port priorities, and key. A member joins the bundle only when both ends agree it should. Miscabling produces a port that stays out of the bundle — a fault that is visible, contained, and safe.

Part F — Priorities decide the active links.

LACP allows more member ports than active links. The rest wait as backups.

```
lacp priority 100                # system priority; lower wins
interface Eth-Trunk 1
  max active-linknumber 2
quit

interface GigabitEthernet 0/0/1
  lacp priority 100              # port priority; lower is preferred
quit
```

Key idea

Two priorities, two different jobs.

System priority decides which switch is in charge of selecting active links. Lower wins; ties break on system MAC. Only one end gets to choose, otherwise the two could disagree.

Port priority is then used by that switch to rank candidate ports. Lower is preferred.

Setting port priorities on the switch that lost the system-priority election has no effect — a result students often mistake for a broken configuration.

Verify which end is deciding, and which ports are selected:

```
<SW1> display eth-trunk 1
<SW1> display lacp statistics eth-trunk 1
```

When it does not work

Trunk stays down. Member ports differ in speed or duplex. All members must match.

Members configured but not selected. Mode mismatch — one end manual, the other LACP. Both ends must run the same mode.

One port never becomes active. Expected if `max active-linknumber` is lower than the member count. Confirm it is a backup, not a fault.

Port refuses to join. It probably still carries per-interface configuration. Ports must be cleared before being added to a trunk.

Try it yourself

With the trunk up, shut down one active member and watch `display eth-trunk 1`. Does a backup take over, and how long does it take?

Then, on the switch that lost the system-priority election, change a port priority and confirm nothing happens. Explain why in one sentence.

Verification evidence

- Both manual and LACP modes configured and captured separately.
- Both ends use the same mode.
- `display eth-trunk 1` shows members Selected, with the count matching `max active-linknumber`.
- The deciding switch is identified from system priority.
- Failover to a backup member is demonstrated and timed.

Lab deliverable

Submit both configurations, `display eth-trunk` output for each mode, evidence of member failover with timing, and a short note explaining why LACP is preferred to manual mode — referring specifically to the miscabling scenario.

VLAN Configuration: Access, Trunk, and Hybrid Ports

Lab at a glance

Before you start: Ethernet switch configuration and Eth-Trunk/LACP concepts from Lab 7.

By the end, you can: Create VLANs, bind access ports, turn the inter-switch Eth-Trunk into a VLAN trunk, configure hybrid ports and PVID behavior, and verify same-VLAN and controlled cross-VLAN communication.

8.1 Purpose of VLANs

VLANs create separate Layer-2 broadcast domains on shared switches. Hosts in different VLANs remain isolated unless the design deliberately connects them.

Source visual — VLAN topology from the supplied Lab 8 slide

Topology

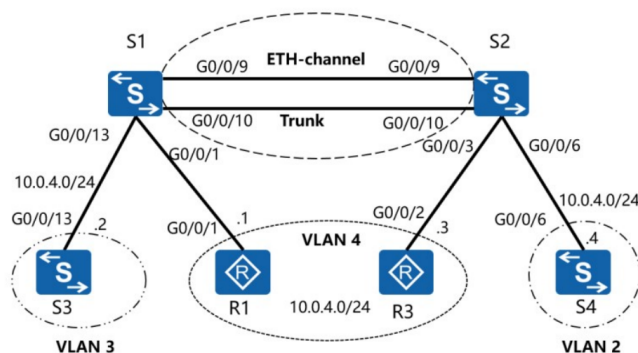
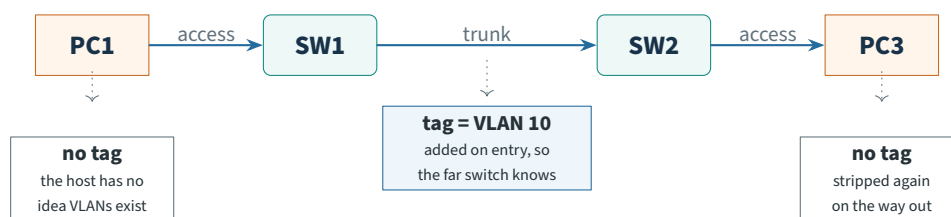


Figure 1.2 VLAN topology

8.2 Three port behaviors used in the exercise

Port type	Role in the supplied lab
Access	Connects an untagged host to one VLAN.
Trunk	Carries multiple VLANs between switches.
Hybrid	Controls tagging per VLAN and classifies untagged ingress by PVID.

Where the VLAN tag is added, carried, and removed



The tag exists only *between* the switches. Hosts never see it, which is why a VLAN needs no configuration on the PC — and why the switch, not the host, decides which VLAN a frame belongs to.

Key idea

A VLAN is enforced by the switch, not by the host's IP address. Two PCs can share a subnet and still be unable to reach each other if they sit in different VLANs — changing the PC's address cannot bypass the separation.

8.3 Step 1: build the inter-switch Eth-Trunk

Build static-LACP Eth-Trunk 1 with G0/0/9 and G0/0/10. The source shows both physical-interface membership and trunkport from the Eth-Trunk view.

```

[S1] interface Eth-Trunk 1
[S1-Eth-Trunk1] mode lacp
[S1] interface GigabitEthernet 0/0/9
[S1-GigabitEthernet0/0/9] eth-trunk 1
[S1] interface GigabitEthernet 0/0/10
[S1-GigabitEthernet0/0/10] eth-trunk 1

[S2] interface Eth-Trunk 1
[S2-Eth-Trunk1] mode lacp
[S2-Eth-Trunk1] trunkport GigabitEthernet 0/0/9
[S2-Eth-Trunk1] trunkport GigabitEthernet 0/0/10
  
```

8.4 Step 2: make Eth-Trunk 1 a VLAN trunk

```

[S1] interface Eth-Trunk 1
[S1-Eth-Trunk1] port link-type trunk
[S1-Eth-Trunk1] port trunk allow-pass vlan all

[S2] interface Eth-Trunk 1
[S2-Eth-Trunk1] port link-type trunk
[S2-Eth-Trunk1] port trunk allow-pass vlan all
  
```

Key idea

Eth-Trunk aggregates links; trunk mode carries VLANs. These are separate functions on the same logical interface.

8.5 Step 3: create VLANs and bind access interfaces

The lab demonstrates two Huawei VRP configuration styles.

On S1, G0/0/13 is associated with VLAN 3 and G0/0/1 with VLAN 4:

```

[S1] interface GigabitEthernet 0/0/13
[S1-GigabitEthernet0/0/13] port link-type access
  
```

```
[S1] interface GigabitEthernet 0/0/1
[S1-GigabitEthernet0/0/1] port link-type access
[S1] vlan 2
[S1-vlan2] vlan 3
[S1-vlan3] port GigabitEthernet 0/0/13
[S1-vlan3] vlan 4
[S1-vlan4] port GigabitEthernet 0/0/1
```

On S2, the source uses `vlan batch` and `port default vlan`:

```
[S2] vlan batch 2 to 4
[S2] interface GigabitEthernet 0/0/3
[S2-GigabitEthernet0/0/3] port link-type access
[S2-GigabitEthernet0/0/3] port default vlan 4
[S2] interface GigabitEthernet 0/0/6
[S2-GigabitEthernet0/0/6] port link-type access
[S2-GigabitEthernet0/0/6] port default vlan 2
```

8.6 Step 4: address the hosts and verify VLAN isolation

Hosts in the same VLAN use a common subnet. Initially, R1 and R3 in VLAN 4 should communicate; hosts in different VLANs should not.

Source visual — Ping evidence used in the supplied VLAN verification sequence

Effectivity
 VLAN 4 should be able to communicate with one another.
 R1 and R3 should be able to communicate.

```
[R1]ping 10.0.4.4
PING 10.0.4.4: 56 data bytes, press CTRL_C to break
Request time out
Request time out
Request time out
Request time out
Request time out

--- 10.0.4.4 ping statistics ---
5 packet(s) transmitted
0 packet(s) received
```

Verification evidence

Test both outcomes: same-VLAN connectivity and different-VLAN isolation before adding hybrid behavior.

8.7 Step 5: change selected ports to hybrid mode

Hybrid ports then deliver VLAN 2 and VLAN 4 traffic untagged to selected devices.

```
[S1] interface GigabitEthernet 0/0/1
[S1-GigabitEthernet0/0/1] port link-type hybrid
[S1-GigabitEthernet0/0/1] port hybrid untagged vlan 2 4
[S1-GigabitEthernet0/0/1] port hybrid pvid vlan 4
```



```
[S2] interface GigabitEthernet 0/0/3
[S2-GigabitEthernet0/0/3] port link-type hybrid
[S2-GigabitEthernet0/0/3] port hybrid untagged vlan 2 4
[S2-GigabitEthernet0/0/3] port hybrid pvid vlan 4

[S2] interface GigabitEthernet 0/0/6
[S2-GigabitEthernet0/0/6] port link-type hybrid
[S2-GigabitEthernet0/0/6] port hybrid untagged vlan 2 4
[S2-GigabitEthernet0/0/6] port hybrid pvid vlan 2
```

8.8 PVID and untagged behavior

The commands are complementary:

- `port hybrid pvid vlan <id>` assigns a VLAN classification to untagged frames received from the host.
- `port hybrid untagged vlan 2 4` permits frames from the listed VLANs to leave that host-facing port without a VLAN tag.

Retest reachability: the selected VLAN-2 and VLAN-4 hosts should communicate, while the VLAN-3 host remains isolated.

Concept map

Ingress from host, no tag → classify using the port PVID.

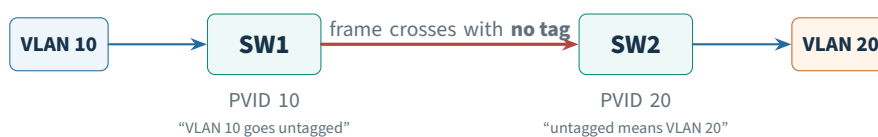
Frame inside the switch → VLAN membership is tracked logically.

Egress toward selected host → listed hybrid VLANs can be sent untagged.

8.9 The native-VLAN mismatch: a leak that reports no error

A trunk sends exactly one VLAN *untagged* — the native VLAN, taken from the port's PVID. If the two ends disagree about which VLAN that is, each switch files the untagged frames into a different VLAN, and two supposedly isolated VLANs are joined.

Mismatched native VLAN silently merges two VLANs



Result: VLAN 10 traffic is delivered into VLAN 20.
Both switches log nothing. Both configurations look correct.

Common mistake

If a cross-VLAN ping unexpectedly *succeeds*, that is not a success — it is this bug. Set the native VLAN explicitly, to an unused VLAN carrying no user data (999 in the lab files), on *both* ends. Confirm with `display port vlan` rather than assuming, because nothing else will tell you.

8.10 Troubleshooting VLAN reachability

Step 1: Check the host IP/subnet first; VLAN experiments still require correct Layer-3 addressing on the endpoints.

Step 2: Confirm the host-facing port link type and VLAN/PVID assignment.

Step 3: Confirm the VLAN exists on both switches where required.

Step 4: Confirm Eth-Trunk 1 is up and is configured as a trunk carrying the relevant VLANs.

Step 5: For hybrid ports, inspect both the PVID and the tagged/untagged VLAN lists.

Step 6: Use ping results as evidence only after the switch configuration matches the intended VLAN path.

Try it yourself

Complete the source assignment: build a topology with two switches, two routers, and two PCs. Put the routers in VLAN 2 and the PCs in VLAN 3. Verify same-VLAN and different-VLAN connectivity and capture the command-prompt evidence.

Check your understanding

1. Why is the inter-switch Eth-Trunk configured as a trunk rather than an access port?
2. What does an access port simplify for a non-VLAN-aware host?
3. In the source exercise, what does the hybrid PVID control?
4. Why should the lab include both successful and failed ping tests?

Lab recap

Access ports serve one VLAN, trunks carry several, and hybrid ports control ingress classification and egress tagging per VLAN. Verify the design with both successful and failed connectivity tests.

PRACTICAL MILESTONE

Lab 8 of 9

VLANs: one switch, several networks

Access, trunk, and hybrid ports — and what each does to a tag

Coverage. Chapter 8.

Part A — What a VLAN actually separates.

A switch floods broadcasts to every port. A VLAN restricts that flooding to a subset, which means one physical switch behaves as several independent switches.

Key idea

Devices in different VLANs cannot reach each other *at all* without a router, regardless of IP addressing. Putting them in the same subnet changes nothing — the frames never leave their broadcast domain.

That is the point. VLAN separation is enforced at layer 2, below IP, so it cannot be bypassed by reconfiguring a host.

Part B — Three port types.

Port type	Incoming untagged frame	Outgoing frame
Access	Tagged with the port's PVID	Tag removed
Trunk	Tagged with PVID	Tagged, unless the VLAN is the native VLAN
Hybrid	Tagged with PVID	Tagged <i>or</i> untagged, configured per VLAN

Key idea

Every port has a **PVID** — the VLAN assigned to arriving untagged frames. For an access port the PVID is simply its VLAN.

The distinction that matters: access and trunk ports decide tagging by *role*; a hybrid port decides per VLAN. That is what lets a hybrid port send VLAN 10 untagged to a PC while sending VLAN 20 tagged to a phone on the same cable.

Part C — Create VLANs and assign access ports.

```
system-view
vlan batch 10 20 30

interface GigabitEthernet 0/0/1
port link-type access
port default vlan 10
quit

interface GigabitEthernet 0/0/2
port link-type access
port default vlan 20
quit
```

Part D — Carry VLANs between switches.

An access port carries one VLAN. Connecting two switches that share three VLANs would need three cables — unless the link is a trunk.

```
interface Eth-Trunk 1
  port link-type trunk
  port trunk allow-pass vlan 10 20 30
quit
```

Common mistake

`port trunk allow-pass vlan 10 20 30` does not *add* to the allowed list on every VRP version — on some it replaces it. Always confirm with `display port vlan` rather than assuming your second command extended the first.

And note that VLAN 1 is allowed by default. Leaving it there is a common audit finding, since it carries management traffic across a link that may not need it.

Part E — The native VLAN trap.**Key idea**

A trunk sends one VLAN untagged: the **native VLAN**, which is the trunk port's PVID and defaults to VLAN 1.

If the two ends of a trunk disagree about the native VLAN, traffic leaks silently between them. Switch A sends VLAN 1 untagged; switch B receives an untagged frame and assigns it to *its* native VLAN, say VLAN 99. Two supposedly separate VLANs are now joined, with no error message anywhere.

Set the native VLAN explicitly and identically on both ends, and never use a VLAN that carries user data.

```
interface Eth-Trunk 1
  port trunk pvid vlan 999
  port trunk allow-pass vlan 10 20 30
quit
```

Part F — Hybrid ports.

```
interface GigabitEthernet 0/0/5
  port link-type hybrid
  port hybrid pvid vlan 10
  port hybrid untagged vlan 10
  port hybrid tagged vlan 20
quit
```

A PC sending untagged frames lands in VLAN 10 and receives VLAN 10 untagged. A device that understands tags can also use VLAN 20 on the same port.

Part G — Verify.

Command map

Command	Shows
<code>display vlan</code>	Every VLAN and its member ports
<code>display port vlan</code>	Each port's type, PVID, and allowed VLANs
<code>display vlan 10 verbose</code>	Which ports carry VLAN 10, tagged or untagged
<code>display mac-address vlan 10</code>	Which hosts have been learned in that VLAN

Then test: hosts in VLAN 10 must ping each other across both switches, and must fail to reach VLAN 20 even in the same subnet.

When it does not work

Same VLAN, same switch, ping fails. Check both ports are access ports in that VLAN with `display port vlan`.

Same VLAN, different switches, ping fails. The trunk is not allowing that VLAN, or the VLAN does not exist on the second switch. It must be created on both.

Different VLANs can reach each other. Native VLAN mismatch on the trunk — the leak described in Part E.

Trunk works for some VLANs only. `allow-pass` replaced rather than extended the list.

Try it yourself

Set the native VLAN to 10 on one switch and 20 on the other, then ping between VLANs. It will work — which is a security failure, not a success.

Capture it in Wireshark and explain which frames carry a tag and which do not. Then fix it and confirm the isolation returns.

Verification evidence

- VLANs created on both switches, not just one.
- Trunk explicitly permits every VLAN that must cross it.
- Native VLAN set identically on both ends and carries no user data.
- Same-VLAN connectivity proven across switches, and cross-VLAN isolation proven to fail.
- Hybrid port demonstrated with one VLAN tagged and one untagged.

Lab deliverable

Submit both switch configurations, `display vlan` and `display port vlan` output, a connectivity matrix showing which host pairs can reach each other and which cannot, and evidence of the native-VLAN mismatch experiment with an explanation of why it is a security problem.

WAN Links: HDLC, PPP, PAP, and CHAP

Lab at a glance

Before you start: Routing, OSPF, IPv4 interface addressing, and basic AAA concepts from earlier labs.

By the end, you can: Configure serial addressing, apply HDLC and PPP encapsulation, identify a DCE and set its baud rate, run OSPF across WAN links, configure PAP and CHAP authentication, and use PPP debugging to investigate negotiation.

9.1 Scenario: connect headquarters and branches

R2 represents headquarters and R1/R3 the branches. Two serial links share one routing domain; the lab moves from HDLC to PPP, then adds PAP and CHAP.

Source visual — Serial WAN topology from the supplied Lab 9 slide

Topology

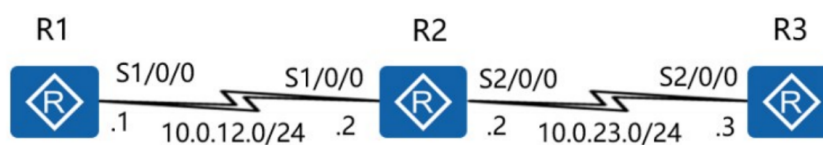


Figure 2.1 HDLC and PPP configuration topology

Link	Interfaces	Network	Addressing shown
R1–R2	S1/0/0 – S1/0/0	10.0.12.0/24	R1 .1, R2 .2
R2–R3	S2/0/0 – S2/0/0	10.0.23.0/24	R2 .2, R3 .3

9.2 HDLC and PPP in the source material

HDLC and PPP are serial link encapsulations that must match at both ends. PPP also supports network-layer negotiation and authentication through PAP or CHAP.

Feature	HDLC portion	PPP portion
Role in lab	Establish the serial Layer-2 link	Replace HDLC and add authentication capability
End agreement	Both ends use HDLC	Both ends use PPP
Authentication	Not used in the HDLC section	PAP between R1–R2 and CHAP between R2–R3

9.3 Step 1: configure serial IP addressing

```
[R1] interface Serial 1/0/0
[R1-Serial1/0/0] ip address 10.0.12.1 24

[R2] interface Serial 1/0/0
[R2-Serial1/0/0] ip address 10.0.12.2 24
[R2] interface Serial 2/0/0
[R2-Serial2/0/0] ip address 10.0.23.2 24

[R3] interface Serial 2/0/0
[R3-Serial2/0/0] ip address 10.0.23.3 24
```

9.4 Step 2: enable HDLC on both serial links

```
[R1] interface Serial 1/0/0
[R1-Serial1/0/0] link-protocol hdlc
[R2] interface Serial 1/0/0
[R2-Serial1/0/0] link-protocol hdlc
[R2] interface Serial 2/0/0
[R2-Serial2/0/0] link-protocol hdlc
[R3] interface Serial 2/0/0
[R3-Serial2/0/0] link-protocol hdlc
```

Verify that each serial interface is physically and logically up before adding routing.

9.5 Step 3: run OSPF across the WAN links

Advertise both /24 serial networks in area 0.

```
[R1] ospf 1
[R1-ospf-1] area 0
[R1-ospf-1-area-0.0.0.0] network 10.0.12.0 0.0.0.255

[R2] ospf 1
[R2-ospf-1] area 0
[R2-ospf-1-area-0.0.0.0] network 10.0.12.0 0.0.0.255
[R2-ospf-1-area-0.0.0.0] network 10.0.23.0 0.0.0.255

[R3] ospf 1
[R3-ospf-1] area 0
[R3-ospf-1-area-0.0.0.0] network 10.0.23.0 0.0.0.255
```

After convergence, inspect learned routes and ping from R1 to R3.

9.6 Step 4: identify the DCE and change the clock rate

One side of a synchronous serial link provides clocking. Here R1 Serial1/0/0 is the DCE; change its baud rate from 64,000 to 128,000 bit/s.

Source visual — Serial interface detail used to identify DCE and clocking

```
[R1]display interface Serial1/0/0
Serial1/0/0 current state : UP
Line protocol current state : UP
Last line protocol up time : 2016-3-10 11:25:08
Description:HUAWEI, AR Series, Serial1/0/0 Interface
Route Port,The Maximum Transmit Unit is 1500, Hold timer is 10(sec)
Internet Address is 10.0.12.1/24
Link layer protocol is nonstandard HDLC
Last physical up time   : 2016-3-10 11:23:55
Last physical down time : 2016-3-10 11:23:55
Current system time: 2016-3-10 11:25:46
Physical layer is synchronous, Baudrate is 64000 bps
Interface is DCE, Cable type is V24, Clock mode is DCECLK
Last 300 seconds input rate 3 bytes/sec 24 bits/sec 0 packets/sec
Last 300 seconds output rate 3 bytes/sec 24 bits/sec 0 packets/sec

Input: 100418 packets, 1606804 bytes
Broadcast:      0, Multicast:      0
Errors:         0, Runts:          0
Giants:         0, CRC:           0

Alignments:     0, Overruns:      0
```

```
[R1] interface Serial 1/0/0
[R1-Serial1/0/0] baudrate 128000
```

Key idea

Set the baud rate only on the DCE side, identified from the interface output.

9.7 Step 5: replace HDLC with PPP

Change both ends of each serial link to PPP.

```
[R1] interface Serial 1/0/0
[R1-Serial1/0/0] link-protocol ppp
[R2] interface Serial 1/0/0
[R2-Serial1/0/0] link-protocol ppp
[R2] interface Serial 2/0/0
[R2-Serial2/0/0] link-protocol ppp
[R3] interface Serial 2/0/0
[R3-Serial2/0/0] link-protocol ppp
```

An encapsulation mismatch leaves the line protocol down; configure both ends before testing.

9.8 Step 6: configure PAP on R1–R2

R1 authenticates R2 with PAP.

```
[R1] interface Serial 1/0/0
[R1-Serial1/0/0] ppp authentication-mode pap
[R1] aaa
[R1-aaa] local-user huawei password cipher huawei123
[R1-aaa] local-user huawei service-type ppp
```



```
[R2] interface Serial 1/0/0
[R2-Serial1/0/0] ppp pap local-user huawei password cipher huawei123
```

Then retest the R1–R2 link.

9.9 Step 7: configure CHAP on R2–R3

The source text names R1 here, but its topology and commands identify R3 as the authenticator and R2 as the CHAP peer. The sequence below follows the commands.

```
[R3] interface Serial 2/0/0
[R3-Serial2/0/0] ppp authentication-mode chap
[R3] aaa
[R3-aaa] local-user huawei password cipher huawei123
[R3-aaa] local-user huawei service-type ppp

[R2] interface Serial 2/0/0
[R2-Serial2/0/0] ppp chap user huawei
[R2-Serial2/0/0] ppp chap password cipher huawei123
```

9.10 Step 8: observe CHAP negotiation with debugging

Cycle the serial interface and enable PPP CHAP debugging to regenerate negotiation messages and locate authentication failure.

Source visual — PPP CHAP debugging output shown in the supplied Lab 9 sequence

Information is displayed

```

21
54 e5 e2 a1 ed 90 87

[R2-Serial2/0/0]
Mar 10 2016 09:10:38.710.4+00:00 R2 PPP/7/debug2:
  PPP State Change:
    Serial2/0/0 CHAP : ListenChallenge --> SendResponse
[R2-Serial2/0/0]
Mar 10 2016 09:10:38.720.1+00:00 R2 PPP/7/debug2:
  PPP Packet:
    Serial2/0/0 Input  CHAP(c223) Pkt, Len 20
    State SendResponse, code SUCCESS(03), id 1, len 16
    Message: Welcome to .
[R2-Serial2/0/0]
Mar 10 2016 09:10:38.720.2+00:00 R2 PPP/7/debug2:
  PPP Event:
    Serial2/0/0 CHAP Receive Success Event
    state SendResponse
[R2-Serial2/0/0]
Mar 10 2016 09:10:38.720.3+00:00 R2 PPP/7/debug2:
  PPP State Change:
27
```

Verification evidence

Verify in order: **physical state** → **DCE clocking** → **encapsulation** → **IP** → **OSPF** → **authentication** → **ping**.

9.11 Troubleshooting matrix

Symptom	First evidence to inspect
Physical state down	Cabling/serial connection and device/interface state.
Physical up, protocol down	HDLC/PPP encapsulation match, DCE clocking, and PPP authentication if enabled.
Direct ping works, remote ping fails	OSPF neighbor/routes and the destination routing table.
PPP works until authentication is added	PAP/CHAP mode, username, password, AAA local user and service type.
CHAP result unclear	Reinitialize the interface as directed and inspect PPP CHAP debug output.

Common mistake

An authentication problem can make the link protocol fail even when the cable and IP address appear correct. Once PAP or CHAP is enabled, the username/password/AAA configuration becomes part of Layer-2 link establishment and must be checked alongside encapsulation.

Try it yourself

Complete the source assignment: create a three-router WAN using PPP, configure PAP and CHAP authentication on the required links, verify connectivity, and capture command-prompt evidence that demonstrates the working links.

Check your understanding

1. Why must both ends of a serial link agree on HDLC or PPP encapsulation?
2. What is the purpose of the DCE side in the supplied serial exercise?
3. Which link uses PAP and which uses CHAP in the lab?
4. Why does the debugging procedure cycle the interface before observing CHAP messages?

Lab recap

Diagnose serial state, DCE clocking, encapsulation, IP routing, and PPP authentication in that order, using display or debug evidence at each stage.

PRACTICAL MILESTONE

Lab 9 of 9

WAN links: HDLC, PPP, and authentication

Connect two sites over serial, then make each end prove who it is

Coverage. Chapter 9.

Goal. Configure serial encapsulation, switch from HDLC to PPP, and compare PAP with CHAP by capturing both.

Part A — Serial links are not Ethernet.

Two differences matter immediately.

There is no MAC addressing and no ARP — a serial link is point-to-point, so there is only one possible recipient. And one end must supply clocking.

Key idea

On a serial link, one end is **DCE** and provides the clock; the other is **DTE** and follows it. In eNSP the DCE end is determined by which side you attached the cable from, and it is not obvious by inspection.

Find it before troubleshooting anything else:

```
<R1> display interface Serial 0/0/0
```

No clock rate on the DCE end means the line stays down no matter how correct the IP configuration is.

```
interface Serial 0/0/0
 ip address 10.0.0.1 30
 clock rate 64000          # DCE end only
 undo shutdown
 quit
```

Part B — HDLC.

```
interface Serial 0/0/0
 link-protocol hdlc
 quit
```

Common mistake

Both ends must run the same encapsulation. HDLC on one end and PPP on the other gives Physical up, Protocol down — the exact pattern from Lab 2.6, and the reason that lab insisted you learn to read those two columns separately.

Note also that HDLC is vendor-specific in practice. Huawei's and Cisco's implementations do not interoperate, which is one reason PPP is the safer default between equipment from different vendors.

Part C — Run OSPF across the WAN.

Bring up OSPF over the serial links using Lab 4.8's configuration, and then look at the cost:

```
<R1> display ip routing-table protocol ospf
```

The serial path carries a far higher cost than any Ethernet path, because Huawei derives cost from bandwidth. If a parallel Ethernet route exists, OSPF will prefer it automatically — which is usually correct, and occasionally not what you intended.

Part D — Switch to PPP.

```
interface Serial 0/0/0
  link-protocol ppp
quit
```

VRP asks for confirmation because changing encapsulation drops the link. Change both ends.

Key idea

PPP brings up a link in two phases, and knowing which phase failed halves your troubleshooting.

LCP negotiates the link itself: MTU, magic numbers, and whether authentication is required.

NCP then negotiates each network-layer protocol, IPCP for IPv4.

Authentication sits between them. So a link that completes LCP and never reaches NCP is almost always failing authentication, not addressing.

Part E — PAP.

On the authenticating side:

```
aaa
  local-user branch password cipher Branch@123
  local-user branch service-type ppp
quit

interface Serial 0/0/0
  ppp authentication-mode pap
quit
```

On the side being authenticated:

```
interface Serial 0/0/0
  ppp pap local-user branch password cipher Branch@123
quit
```

Part F — CHAP, and why it is better.

```
interface Serial 0/0/0
  ppp authentication-mode chap
quit

# far end
interface Serial 0/0/0
  ppp chap user branch
  ppp chap password cipher Branch@123
quit
```

Key idea

PAP sends the password across the link in plaintext, once, at setup. Anyone capturing the link has the credential.

CHAP never sends the password. The authenticator sends a random challenge; the peer replies with a hash of the challenge combined with the secret. The authenticator computes the same hash and compares. A captured exchange yields nothing reusable, because the next challenge differs.

CHAP also re-challenges periodically during the session, so a link cannot be hijacked after setup.

Part G — Prove the difference.

Capture both. With PAP the password is readable in the capture; with CHAP you see only a challenge and a hash.

Command map

Command	Shows
<code>display interface Serial 0/0/0</code>	Encapsulation, clock, LCP state, DCE/DTE
<code>display ppp all</code>	PPP state per interface
<code>debugging ppp all</code>	Live negotiation — use sparingly
<code>display ip interface brief</code>	Whether the link came up at all

When it does not work

Physical down. Cable not connected in eNSP, or the device is not started.

Physical up, protocol down. Encapsulation mismatch, missing clock rate on the DCE end, or failed authentication. Check in that order.

LCP completes, IPCP never does. Authentication is failing. The username must match the `local-user` exactly, and the password on both sides.

Works with PAP, fails with CHAP. CHAP matches on the hostname by default. Confirm the `ppp chap user` matches the configured `local-user`.

Try it yourself

Deliberately misconfigure the CHAP password on one end and capture the exchange. How many messages appear before the link gives up, and does the failure say anything useful? Then relate the answer to Lab 5.9: both labs show a credential crossing a link. Explain which design is safer and why, in two sentences.

Verification evidence

- DCE end identified from `display interface`, with the clock rate set there.
- Both HDLC and PPP configured, each with its own captured output.
- Both PAP and CHAP demonstrated with matching credentials.
- Captures show the PAP password in plaintext and the CHAP challenge-response.
- OSPF adjacency reaches Full across the serial link.

Lab deliverable

Submit configurations for both routers in each encapsulation, `display interface Serial` output showing the DCE end and protocol state, packet captures of both PAP and CHAP with the security difference annotated, and the OSPF routing table showing the serial link's cost relative to Ethernet.

Group Project: Integrated eNSP Network

Source basis

Based on **Group Project - Template DCN.ppt**.

What the project asks you to demonstrate

Build a client-server scenario in eNSP and explain it through **two or three OSI/TCP-IP layers**.

Layer/theme	Project task from the supplied template
Application	Configure a client-server service in eNSP.
Transport	Demonstrate TCP connection setup, rate limiting, packet loss, and retransmission.
Network	Build 2–3 routers with multiple hosts, IPv4 addressing, static routing, and packet forwarding.
Data Link (optional)	Explain how frames cross the selected physical or logical link.

A build sequence that reuses the lab workflow

Step 1: Draw the topology and write the addressing plan before entering any CLI commands.

Step 2: Configure host/router interfaces and prove every directly connected link.

Step 3: Add static routes and verify each remote destination in the routing table.

Step 4: Prove end-to-end forwarding before adding the client-server service.

Step 5: Add the chosen application/TCP behavior and collect packet or command evidence.

Step 6: If using Data Link, show its effect on the end-to-end flow.

Step 7: Label each readable screenshot and tie it to a technical claim.

Recommended evidence matrix

Claim in presentation/report	Strong evidence
“The topology is addressed correctly”	Addressing table plus <code>display ip interface brief</code> /host IP configuration.
“Routers can forward between networks”	Routing-table entries and successful end-to-end ping.
“TCP provides reliable transfer”	Capture of connection setup and retransmission after loss.
“Rate limiting affects the flow”	Before/after measurements tied to the configured limit.
“The design uses a specific layer/protocol”	Configuration excerpt plus a packet/state observation that proves the protocol is active.

Deliverables

The project has three outputs:

- **eNSP demo:** a working network explained while it runs.
- **Presentation:** design, key configuration, protocol choices, and evidence.
- **Report:** a record of the implementation. The source states Week 12 submission through uLearn and to the lab instructor.

Evaluation criteria

The rubric weights are:

Design & Implementation	40%
Functionality & Demo	30%
Presentation & Explanation	20%
Report Quality	10%

Bands are Excellent (90–100), Good (75–89), Satisfactory (60–74), and Needs Improvement (<60).

Source visual — Evaluation-criteria table from the supplied project template

Evaluation Criteria

Criteria	Excellent (90-100%)	Good (75-89%)	Satisfactory (60-74%)	Needs Improvement (<60%)
Design & Implementation (40%)	Clear, logical, accurate configuration	Mostly clear with minor errors	Basic design with major errors	Incomplete or incorrect setup
Functionality & Demo (30%)	Fully functional, end-to-end communication	Mostly functional with minor issues	Partial functionality	Minimal or no functionality
Presentation & Explanation (20%)	Well-explained, confident delivery	Clear but lacks depth	Basic explanations	Poorly explained, unclear
Report Quality (10%)	Comprehensive, well-written	Mostly complete, minor gaps	Basic coverage, lacks depth	Incomplete, unclear

Key idea

Pair each claim with evidence: interface state, a route, a packet trace, an application result, or a connectivity test.

Lab deliverable

Submit a topology, addressing table, configuration excerpts, routing and TCP evidence, concise layer explanations, and brief troubleshooting/limitations notes.

Command Quick Reference

This appendix is a lookup aid. Confirm the required VRP view before entering a command.

Core VRP and interface commands

Command	Purpose in the labs
<code>system-view</code>	Enter system configuration view.
<code>sysname R1</code>	Rename the device.
<code>interface GigabitEthernet 0/0/0</code>	Enter an Ethernet interface view.
<code>ip address 10.0.13.1 24</code>	Assign an IPv4 address/prefix.
<code>description <text></code>	Label an interface.
<code>display this</code>	Display the configuration in the current view.
<code>display interface <interface></code>	Display detailed interface state.
<code>display ip interface brief</code>	Compact IP/interface state summary.
<code>ping <address></code>	Test IP reachability.
<code>save</code>	Save the current configuration.
<code>display saved-configuration</code>	Display the stored configuration.

Static routing and OSPF

Command	Purpose
<code>ip route-static 10.0.3.0 24 10.0.23.3</code>	Add a next-hop static route.
<code>ip route-static ... preference 80</code>	Add a less-preferred backup static route.
<code>display ip routing-table</code>	Inspect direct, static, and dynamic routes.
<code>ospf 1 router-id 10.0.1.1</code>	Create/select OSPF process 1 with a router ID.
<code>area 0</code>	Enter OSPF area 0.
<code>network 10.0.12.0 0.0.0.255</code>	Enable matching interfaces for OSPF.
<code>display ospf peer</code>	Detailed OSPF neighbor state.
<code>display ospf peer brief</code>	Condensed OSPF neighbor summary.

FTP

Command	Purpose
<code>ftp server enable</code>	Enable the FTP server.
<code>set default ftp-directory flash:/</code>	Set default server directory.
<code>local-user huawei ...</code>	Create/configure the local FTP/PPP user under AAA.
<code>display ftp-server</code>	Verify server status and port.
<code>ftp 10.0.12.1</code>	Open an FTP client connection.
<code>dir</code>	List files in the FTP session.
<code>binary</code>	Set binary transfer mode.
<code>get <remote> <local></code>	Download from server to client.
<code>put <local> <remote></code>	Upload from client to server.
<code>bye</code>	Close the FTP session.

DHCP

Command	Purpose
<code>dhcp enable</code>	Enable DHCP functionality.
<code>ip pool pool1</code>	Create a named global pool.
<code>network 10.0.12.0 mask 24</code>	Define pool network.
<code>gateway-list 10.0.12.1</code>	Define default gateway offered by pool.
<code>lease day 1 hour 12</code>	Configure lease duration.
<code>dhcp select global</code>	Use a global pool on an interface.
<code>dhcp select interface</code>	Use an interface-based pool.
<code>dhcp server excluded-ip-address <IP></code>	Prevent an address from dynamic allocation.
<code>dhcp server dns-list <IP></code>	Offer a DNS server value.
<code>ip address dhcp-alloc</code>	Make a switch Vlanif request an address dynamically.
<code>display ip pool name <name></code>	Inspect pool state and lease counters.

Eth-Trunk, LACP, and VLAN

Command	Purpose
<code>undo negotiation auto</code>	Disable Ethernet auto-negotiation.
<code>speed 100</code>	Force the lab member-link speed.
<code>interface Eth-Trunk 1</code>	Enter/create the logical bundle.
<code>eth-trunk 1</code>	Add a physical interface to Eth-Trunk 1.
<code>mode lacp</code>	Enable static LACP on the Eth-Trunk (syntax varies by VRP image).
<code>lacp priority 100</code>	Set LACP system/interface priority depending on view.
<code>port link-type access</code>	Configure a host-facing access port.
<code>port default vlan 4</code>	Bind an access port to VLAN 4.
<code>port link-type trunk</code>	Configure trunk behavior.
<code>port trunk allow-pass vlan all</code>	Permit VLANs on the trunk.
<code>port link-type hybrid</code>	Configure a hybrid port.
<code>port hybrid untagged vlan 2 4</code>	Send listed VLANs untagged on egress.
<code>port hybrid pvid vlan 4</code>	Classify untagged ingress using VLAN 4 as PVID.

WAN, PPP, PAP, and CHAP

Command	Purpose
<code>interface Serial 1/0/0</code>	Enter a serial interface.
<code>link-protocol hdlc</code>	Use HDLC encapsulation.
<code>baudrate 128000</code>	Change DCE clock rate in the lab.
<code>link-protocol ppp</code>	Use PPP encapsulation.
<code>ppp authentication-mode pap</code>	Require PAP authentication.
<code>ppp pap local-user huawei password cipher ...</code>	Configure PAP credentials on the authenticated peer.
<code>ppp authentication-mode chap</code>	Require CHAP authentication.
<code>ppp chap user huawei</code>	Configure CHAP username.
<code>ppp chap password cipher ...</code>	Configure CHAP password.

Common mistake

If a command is rejected, check the prompt and required view before changing its syntax.